# Szakdolgozat

Miskolci Egyetem

# JavaScript alapú webalkalmazások karbantarthatóságának vizsgálata

**Készítette:**

Sándor János

Programtervező informatikus

**Témavezető:**

Dr. Agárdi Anita

Miskolc, 2026

**Miskolci Egyetem**
Gépészmérnöki és Informatikai Kar
Alkalmazott Matematikai Intézeti Tanszék **Szám:**

# Szakdolgozat Feladat

Sándor János (U03GDK) programtervező informatikus jelölt részére.

**A szakdolgozat tárgyköre:** webes alkalmazás fejlesztése

**A szakdolgozat címe:** JavaScript alapú webalkalmazások karbantarthatóságának vizsgálata

**A feladat részletezése:**

*Az alkalmazás fejlesztése JavaScript alapú eszközök kerülnek felhasználásra, mint a React, React Native és Express.js. A szakdolgozat célja, hogy egy (főként szoftverfejlesztési) projektek menedzselésére alkalmas webalkalmazás fejlesztése kapcsán vizsgálja a JavaScript alapú webalkalmazások karbantarthatóságát. A dolgozat kitér a projekt kezelés általános működésére, típusaira. Összehasonlítja az aktuálisan elérhető alternatívákat. Specifikálja a készülő rendszer működését. Az alkalmazás készítése kapcsán kiemelt szerepet kap az alkalmazás forráskódjának karbantarthatósága, figyelembe véve a forráskód szerkezetét és a szoftverfejlesztési módszereket.*

*Az alkalmazás fejlesztéséhez JavaScript alapú eszközök kerülnek felhasználásra, mint a React, React Native és Express.js.*

**Témavezető:** Dr. Agárdi Anita egyetemi adjunktus

**A feladat kiadásának ideje:**

. . . . . .
szakfelelős

# Eredetiségi Nyilatkozat

Alulírott **Sándor János**; Neptun-kód: `U03GDK` a Miskolci Egyetem Gépészmérnöki és Informatikai Karának végzős Programtervező informatikus szakos hallgatója ezennel büntetőjogi és fegyelmi felelősségem tudatában nyilatkozom és aláírásommal igazolom, hogy *JavaScript alapú webalkalmazások karbantarthatóságának vizsgálata* című szakdolgozatom saját, önálló munkám; az abban hivatkozott szakirodalom felhasználása a forráskezelés szabályai szerint történt.

Tudomásul veszem, hogy szakdolgozat esetén plágiumnak számít:

- szószerinti idézet közlése idézőjel és hivatkozás megjelölése nélkül;
- tartalmi idézet hivatkozás megjelölése nélkül;
- más publikált gondolatainak saját gondolatként való feltüntetése.

Alulírott kijelentem, hogy a plágium fogalmát megismertem, és tudomásul veszem, hogy plágium esetén szakdolgozatom visszautasításra kerül.

Miskolc, ..... .év ..... .hó ..... .nap

.....

Hallgató

1.

A szakdolgozat feladat módosítása szükséges (módosítás külön lapon)

nem szükséges

..... .....

dátum témavezető(k)

2. A feladat kidolgozását ellenőriztem:

témavezető (dátum, aláírás): konzulens (dátum, aláírás):

..... .....

..... .....

..... .....

3. A szakdolgozat beadható:

..... .....

dátum témavezető(k)

4. A szakdolgozat ..... szövegoldalt

..... program protokollt (listát, felhasználói leírást)

..... elektronikus adathordozót (részletezve)

.....

..... egyéb mellékletet (részletezve)

.....

tartalmaz.

..... .....

dátum témavezető(k)

5.

A szakdolgozat bírálatra bocsátható

nem bocsátható

A bíráló neve: .....

..... .....

dátum szakfelelős

6. A szakdolgozat osztályzata

a témavezető javaslata: .....

a bíráló javaslata: .....

a szakdolgozat végleges eredménye: .....

Miskolc, ..... .....

a Záróvizsga Bizottság Elnöke

# Tartalomjegyzék


**1. Bevezetés** **1**

**2. Koncepció** **3**
2.1. A fejezet célja . . . 3
2.2. Tartalom és felépítés . . . 3
2.3. Projektmenedzsment rendszerek áttekintése . . . 3
2.3.1. JIRA . . . 3
2.3.2. Trello . . . 4
2.3.3. Asana . . . 4
2.3.4. Redmine . . . 5
2.3.5. Projektmenedzsment rendszerek összehasonlítása . . . 5
2.4. Összegzés . . . 5

**3. Tervezés** **7**
3.1. Rendszerarchitektúra . . . 7
3.2. Technológiai stack . . . 7
3.2.1. Backend . . . 8
3.2.2. Webes frontend . . . 8
3.2.3. Mobil frontend . . . 9
3.3. Karbantarthatósági szempontok a kliens architektúrában . . . 9
3.3.1. Közös állapotkezelési minta . . . 9
3.3.2. Kódduplikáció és annak tudatos kezelése . . . 9
3.3.3. A közös backend mint karbantarthatósági horgony . . . 10
3.3.4. Tesztelhetőség mint karbantarthatósági alap . . . 10
3.3.5. Navigációs architektúra és karbantarthatóság . . . 10
3.4. Adatbázis-tervezés . . . 11
3.4.1. Adatmodellek . . . 11
3.4.2. XP és szintlépési rendszer tervezése . . . 11
3.5. API tervezés . . . 12
3.5.1. Végpont-struktúra . . . 12
3.5.2. Hitelesítési folyamat . . . 13
3.5.3. Feladat-befejezési folyamat . . . 14
3.6. Use case tervezés . . . 14
3.7. Felhasználói felület tervezése . . . 14
3.7.1. Webes alkalmazás elrendezése . . . 15
3.7.2. Mobil alkalmazás elrendezése . . . 15
3.7.3. Nézetek és navigáció . . . 15
3.8. Állapotkezelés tervezése . . . 16

3.9. GitHub integráció tervezése . . . . . 16

**4. Implementáció** **20**
4.1. Backend implementáció . . . . . 20
4.1.1. Projekt struktúra . . . . . 20
4.1.2. Mongoose modellek . . . . . 21
4.1.3. Hitelesítés implementációja . . . . . 21
4.1.4. XP és gamifikációs szolgáltatás . . . . . 22
4.1.5. Naplózási szolgáltatás . . . . . 22
4.1.6. Projektlétrehozás TaskGroup-okkal . . . . . 22
4.2. Webes frontend implementáció . . . . . 23
4.2.1. Könyvtárstruktúra . . . . . 23
4.2.2. Context API és állapotkezelés . . . . . 23
4.2.3. Nézetek implementációja . . . . . 24
4.2.4. Kanban nézet . . . . . 24
4.2.5. Scrum nézet . . . . . 24
4.2.6. Gamifikáció nézet . . . . . 25
4.2.7. Mail rendszer . . . . . 27
4.3. Mobil alkalmazás implementáció . . . . . 27
4.3.1. Navigáció . . . . . 27
4.3.2. Projekt választó . . . . . 28
4.3.3. Alfeladatok nézet . . . . . 28
4.3.4. API kommunikáció . . . . . 28
4.4. Tesztelés implementációja . . . . . 28
4.4.1. Backend tesztelés . . . . . 28
4.4.2. Frontend tesztelés . . . . . 29
4.4.3. GitHub service tesztelés . . . . . 30
4.5. Fejlesztési kihívások és megoldások . . . . . 31

**5. Tesztelés** **32**
5.1. Tesztelési stratégia . . . . . 32
5.2. Backend tesztelés . . . . . 32
5.2.1. Eszközök és konfiguráció . . . . . 32
5.2.2. Auth tesztek . . . . . 33
5.2.3. Projekt tesztek . . . . . 33
5.2.4. Feladat tesztek . . . . . 34
5.2.5. Alfeladat tesztek . . . . . 35
5.2.6. Service tesztek . . . . . 35
5.3. Frontend tesztelés . . . . . 36
5.3.1. Eszközök és konfiguráció . . . . . 36
5.3.2. Reducer tesztek . . . . . 36
5.3.3. Context tesztek . . . . . 36
5.3.4. Action tesztek . . . . . 37
5.3.5. Komponens tesztek . . . . . 37
5.4. Manuális tesztelés . . . . . 37
5.5. Ismert korlátok és nem tesztelt területek . . . . . 39

**6. Összefoglalás** **40**
6.1. A munka összegzése . . . . . 40
6.2. Elért eredmények . . . . . 41
6.3. A karbantarthatóság értékelése . . . . . 41
6.3.1. Kódbázis struktúrájának karbantarthatósága . . . . . 41
6.3.2. Web és mobil kliens összehangoltsága . . . . . 42
6.3.3. Összegző értékelés . . . . . 42
6.4. Jövőbeli fejlesztési lehetőségek . . . . . 42
6.5. Zárszó . . . . . 43

**Irodalomjegyzék** **45**

# 1. fejezet

# Bevezetés

A mindennapi életben végzett tevékenységeink szervezése és nyomon követése az utóbbi években alapvetően megváltozott. A digitalizáció előretörése azt eredményezte, hogy a feladatkezelés már korántsem csupán papíralapú jegyzetfüzetekben, falinaptárakban vagy post-it cédulákon történik. A számítógépek, okostelefonok és felhőalapú szolgáltatások korszakában számos olyan alkalmazás vált széles körben elérhetővé, amelynek célja a feladatok rendszerezése, a határidők figyelése és a személyes hatékonyság növelése. Mégis, a felhasználók jelentős része továbbra is azt tapasztalja, hogy a feladatkezelés nehezen átlátható, túl bonyolult vagy egyszerűen nem illeszkedik a saját gondolkodási folyamataikhoz. Ez a paradoxon – miszerint a rengeteg eszköz ellenére a feladatkezelés nem feltétlenül válik könnyebbé – hívta életre azt az igényt, hogy újragondoljuk, mit is várunk egy modern feladatkezelő rendszertől.

A probléma gyökere abban rejlik, hogy az emberek munkastílusa, gondolkodása és prioritáskezelése rendkívül különböző. Egyesek listákban szeretnek gondolkodni, mások vizuális naptárakban, vannak, akik az egyszerre egy feladatra fókuszálást részesítik előnyben, és olyanok is, akiknek az átlátható projektszerkezet a legfontosabb. A ma elérhető eszközök döntő többsége azonban egy előre meghatározott feladatkezelési filozófiát próbál a felhasználóra erőltetni: vagy listaalapú működést biztosít, vagy kártyaorientált felületet, vagy szigorúan GTD kompatibilis struktúrát, esetleg valamilyen saját módszertant. Ezek az eszközök hatékonyak lehetnek azok számára, akiknek a gondolkodása illeszkedik az adott működésmódhoz, azonban sok felhasználó egyszerűen nem találja meg bennük azt a rugalmasságot és személyre szabhatóságot, amely hosszú távon is fenntartható használatot eredményezne. Emiatt gyakori jelenség, hogy valaki nagy lelkesedéssel kezdi használni az új rendszert, majd néhány hét vagy hónap után teljesen elengedi, mert túl sok munka fenntartani, vagy éppen túl kevés értéket ad a mindennapokhoz.

A feladatkezelők másik általános problémája, hogy sokszor csak felületi változatosságot kínálnak, tartalmi rugalmasságot viszont alig. A feladatokat gyakran ugyanúgy kell megadni: név, határidő, címkék és esetleg valamilyen kategória. Bár ezek alapvető elemek, az emberi gondolkodás ennél jóval gazdagabb. A feladatokhoz kapcsolódhatnak összefüggések, prioritások, előfeltételek, kontextusok, amelyek figyelmen kívül hagyása a rendszer használatát korlátozottá teszi. Emellett az is problémát jelent, hogy a legtöbb eszköz a feladatokat felhasználói szinten egyetlen nagy masszaként kezeli, miközben a felhasználó élete nem egyetlen síkban zajlik. Másképp gondolkodunk a munkáról, a magánéletről, a hosszú távú célokról vagy akár egy hétköznapi bevásárlólistáról.

Ezekre a kihívásokra válaszul született meg az a felismerés, hogy szükség lenne egy

olyan rendszerre, amely nem előre definiált struktúrákat kényszerít a felhasználóra, hanem képes alkalmazkodni hozzá. Egy olyan rendszerre, amely nem csak eszköz, hanem nyelvezet is: amelyben a felhasználó saját gondolkodásához igazíthatja a feladatok és információk szervezésének módját. Ezt az igényt igyekszik kielégíteni az Acxor, amely egy olyan, rugalmas és földre eső feladatkezelési modellt kínál, amely a feladatokat több dimenzió mentén képes értelmezni. Nem egyszerűen listák vagy projektek halmazát kezeli, hanem egymással összefüggő, kontextusba ágyazott elemeket, amelyek tetszőleges módon kombinálhatók.

A rendszer újszerűsége abban rejlik, hogy a feladatokat nem kizárólag hagyományos értelemben vett teendőként kezeli, hanem olyan absztrakciókként, amelyek kapcsolatokat, célokat, állapotokat és prioritásokat is hordozhatnak. A modell lehetővé teszi, hogy a felhasználó egyetlen nézőpont-váltással egészen más jellegű információt kapjon ugyanarról az adathalmazról: láthatja időbeli sorrendben, kontextusok szerint, projektekre bontva vagy akár egyedi szűrők alapján. Az Acxor alapelve az, hogy a rendszernek kell alkalmazkodnia a felhasználóhoz, nem pedig fordítva. Ezzel a megközelítéssel olyan rugalmasság érhető el, amely ritkán található meg a jelenlegi feladatkezelő rendszerekben.

A dolgozat célja ennek a koncepciónak a részletes bemutatása, valamint egy olyan működési modell kidolgozása, amely mind a technikai megvalósítást, mind a felhasználói élményt tekintve életképes és bővíthető alapot biztosít egy későbbi fejlesztéshez. A bevezető fejezet ezért elsősorban azt kívánja megalapozni, hogy miért érdemes ezt a témát kutatni, és milyen gyakorlati igény hívta életre az ötletet. A következő fejezetek majd részletesen bemutatják azokat az elméleti modelleket, amelyekre a koncepció épül, valamint azokat az eszközöket és technikákat, amelyek segítségével egy modern, rugalmas feladatkezelő rendszer megvalósítható. A jelen fejezet feladata inkább az, hogy érzékeltesse a probléma súlyát, és megmutassa, hogy a fejlesztett rendszer milyen társadalmi és technológiai környezetben nyer értelmet.

Összességében tehát a dolgozat egy olyan problémára koncentrál, amely egyszerre technikai, pszichológiai és felhasználói élménybeli kihívás: hogyan lehet olyan feladatkezelő rendszert alkotni, amely valóban segít, és nem újabb terhet jelent? Hogyan lehet egy rendszert úgy felépíteni, hogy az alkalmazkodjon a felhasználó gondolkodásmódjához, és ne kényszerítse bele egy előre meghatározott struktúrába? És hogyan lehet mindezt úgy megtenni, hogy a rendszer mégis átlátható, bővíthető és hosszú távon fenntartható maradjon? E kérdések megválaszolása nem csupán érdekes, hanem egyben aktuális és összetett feladat is, hiszen a digitális produktivitási eszközök környezete gyorsan változik, és a felhasználói igények is folyamatosan fejlődnek.

A dolgozat végső célja olyan modell bemutatása, amely új alapokra helyezi a feladatkezelésről való gondolkodást, és megmutatja, hogy a feladatkezelő rendszerek nem feltétlenül kell, hogy merev és előre meghatározott sémák szerint működjenek. Ezzel nem csupán egy új eszköz koncepcióját vázolja fel, hanem hozzájárul a feladatkezelési rendszerek jövőjéről folytatott szakmai párbeszédhez is.

# 2. fejezet

# Koncepció

## 2.1. A fejezet célja

Ebben a fejezetben a dolgozat témáját körülhatároljuk, bemutatjuk a releváns projektmenedzsment rendszereket, azok főbb funkcióit, előnyeit és hátrányait, valamint azt, hogyan kapcsolódnak az Acxor fejlesztéséhez. A cél nem a saját eredmények részletezése, hanem egy átfogó, irodalmi és technológiai alap bemutatása, amelyre a dolgozat további részei építenek. A hivatkozások jelentős része ebben a fejezetben kerül alkalmazásra, például [**?**].

## 2.2. Tartalom és felépítés

A fejezet tartalma négy fő területre koncentrál: irodalomkutatás, technológia, piackutatás és követelmény specifikáció. Az irodalomkutatás során áttekintjük a legfontosabb projektmenedzsment rendszereket, a technológiai részben bemutatjuk a rendszerek főbb funkcióit, a piackutatás során összehasonlítjuk a meglévő megoldásokat, míg a követelmény specifikációban az Acxor fejlesztés céljait és alapelveit foglaljuk össze.

## 2.3. Projektmenedzsment rendszerek áttekintése

### 2.3.1. JIRA

A JIRA egy komplex, vállalati szintű projektmenedzsment és issue tracking rendszer. Kiemelt jellemzői [**?**]:

- Issue tracking, Agile metodikák (Scrum / Kanban) támogatása
- Workflow testreszabás
- Jogosultságkezelés
- Riportok és dashboardok
- Széleskörű integrációk (Confluence, Bitbucket, GitHub, Slack, Teams)

A JIRA rugalmas és szinte bármi testreszabható, ami előnyt jelent nagyvállalati környezetben, ugyanakkor komplexitása miatt tanulási görbéje meredek. A rendszer

hierarchiája Epic–Story–Task–Subtask struktúrát követ, ami előnyös a nagy csapatok számára. Automatizmusai trigger–condition–action logikán alapulnak, amely lehetővé teszi például automatikus állapotváltást, assignee hozzárendelést, vagy e-mail értesítések küldését.

Az Acxor fejlesztésében a JIRA inspirációként szolgált a funkcionalitás és a workflow kezelés terén, de egyszerűsített felhasználói felületet és átláthatóbb jogosultsági modellt alkalmaz.

### 2.3.2. Trello

A Trello egy intuitív, kártya-alapú projektmenedzsment rendszer, amely kisebb csapatoknak vagy egyszerűbb folyamatokhoz ideális. Főbb jellemzői [**?**]:

- Board → List → Card → Checklist vizuális hierarchia
- Drag & drop alapú kezelés
- Minimális konfiguráció és real-time szinkron
- Egyszerű automatizmusok (Butler)

A Trello előnye a gyors betanulás és a vizuális, átlátható felület, míg hátránya a korlátozott testreszabhatóság és a nagyobb projektekhez kevésbé megfelelő komplexitás. Az Acxor esetében a Trello inspirálta az intuitív, vizuális task lifecycle és board kezelést, amely egyszerre egyszerű és professzionális.

### 2.3.3. Asana

Az Asana egy feladat- és projektmenedzsment platform, amely a csapatok együttműködését támogatja. Fő funkciói [**?**]:

- Task és Subtask kezelés
- Projektek és Timeline vizualizáció
- Határidők és felelősök hozzárendelése
- Dashboard és riporting
- Integráció külső szolgáltatásokkal (Slack, Teams, GitHub, Google Workspace)

Előnye a könnyen átlátható felület és a gyors onboarding, hátránya a korlátozott workflow testreszabás és a részletes riporting hiánya. Az Acxor tervezésénél az Asana inspirációt jelentett a fókuszált task lifecycle és minimalista felület kialakításában.

### 2.3.4. Redmine

A Redmine nyílt forráskódú rendszer, amely elsősorban testreszabhatóságáról és bővíthetőségéről ismert. Főbb jellemzői [?]:

- Issue tracking és projekt hierarchia (főprojekt → alprojekt)
- Testreszabható workflow és permission rendszer
- Gantt és Calendar vizualizáció
- Plugin és API alapú integráció

Előnye a nagy projektek kezelésére való alkalmasság és a nyílt forráskódú bővíthetőség, míg hátránya a kevésbé modern UI és a tanulási görbe. Az Acxorban a Redmine inspirációt jelentett a backend struktúrában és az issue hierarchia egyszerűsítésében, modern, letisztult felülettel.

### 2.3.5. Projektmenedzsment rendszerek összehasonlítása

Az Acxor a közepes komplexitású fejlesztői csapatok számára készült, kiemelten figyelve a vizualitásra, egyszerűségre és motivációs rendszerekre. Összehasonlításképpen bemutatom a Redmine, Asana, Trello és Jira főbb jellemzőit:

| **Funkció / Eszköz** | **Acxor** | **Trello** | **Jira** |
|---|---|---|---|
| Webes felület | Igen, modern vizuális UI | Igen, webes, vizuális | Igen, webes, komplex |
| Kanban board | Igen, beépített, drag&drop | Igen, beépített | Részben, plugin / board szükséges |
| Sprint / Iteration | Részben, egyszerű | Nem | Igen, teljes körű |
| Gamifikáció | Igen, XP, szint, badge | Nem | Nem |
| Task / Subtask | Igen, Epic → Story Task → Subtask | Igen, kártya / checklist | Igen, Epic → Story Task → Subtask |
| Chat / Log | Igen, globális bot | Részben, kommentek | Részben, jegy alapú |
| Workflow testreszabás | Részben, egyszerű | Részben, limitált | Igen, részletes, szabály alapú |
| Permission rendszer | Részben, egyszerű | Részben, egyszerű | Igen, mély, szerepkör alapú |
| Integrációk | Igen, alap, kulcs szolgáltatások | Részben, Power-Ups | Igen, széles (Confluence, GitHub, Slack...) |

2.1. táblázat. Acxor összehasonlítása a Trello-val és a Jira-val

## 2.4. Összegzés

A fenti rendszerek áttekintése rávilágít arra, hogy a projektmenedzsment eszközök széles skálán helyezkednek el komplexitás, testreszabhatóság és felhasználói élmény szempontjából. Az Acxor célja a különböző rendszerek előnyeinek ötvözése: a JIRA funkcionalitásának, a Trello vizuális egyszerűségének, az Asana átlátható task lifecycle-jának, valamint a Redmine struktúrájának kombinálása egy intuitív, könnyen használható, ugyanakkor professzionális projektmenedzsment platformban.

Ez a koncepciói alap biztosítja, hogy a dolgozat későbbi fejezeteiben bemutatott fejlesztések jól meghatározott, átgondolt irányelv szerint történjenek, és az olvasó számára érthető, követhető legyen a választott megközelítés és technológiai háttér.

| **Funkció / Eszköz** | **Acxor** | **Redmine** | **Asana** |
|---|---|---|---|
| Webes felület | Igen, modern vizuális UI | Igen, webes, részletes | Igen, webes, intuitív |
| Kanban board | Igen, beépített | Részben, plugin szükséges | Igen, board / list nézet |
| Sprint / Iteration | Részben, egyszerű | Igen, teljes körű | Részben, idővonal / Timeline |
| Gamifikáció | Igen, XP, szint, badge | Nem | Nem |
| Task / Subtask | Igen, egyszerű hierarchia | Igen, issue / subtask | Igen, task / subtask |
| Chat / Log | Igen, globális bot | Részben, jegy alapú | Részben, kommentek |
| Workflow testreszabás | Részben, egyszerű | Igen, részletes | Részben, egyszerű |
| Permission rendszer | Részben, egyszerű | Igen, roles | Részben, alap |
| Integrációk | Igen, kulcs szolgáltatások | Részben, pluginok, SCM | Igen, Slack, GitHub, Google Workspace |

2.2. táblázat. Acxor összehasonlítása a Redmine-nel és az Asana-val

# 3. fejezet

# Tervezés

A tervezési fejezet célja az Acxor rendszer architektúrájának, adatmodelljének és felhasználói felületének részletes bemutatása. Ebben a fejezetben kerülnek ismertetésre azok a döntések, amelyek a fejlesztés irányvonalát meghatározták, beleértve a technológiai stack kiválasztását, az adatbázis-séma kialakítását, az API tervezését és a komponensstruktúrát. A tervezési fázis különösen fontos volt, hiszen a rendszernek három platformon – webes böngészőben, mobil eszközön és szerver oldalon – egyidejűleg kellett megfelelően működnie, miközben közös üzleti logikát és adatréteget használt.

## 3.1. Rendszerarchitektúra

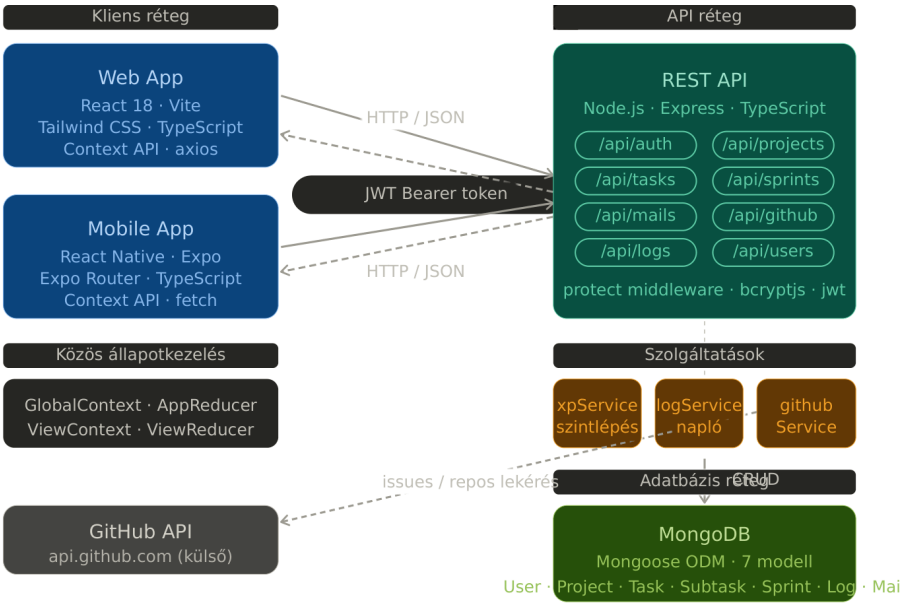
Az Acxor rendszer három jól elkülönülő részből áll: a webes frontend alkalmazásból, a React Native alapú mobil alkalmazásból és a Node.js backend API-ból. Mindhárom rész egyetlen közös RESTful API-on keresztül kommunikál, amelyet a backend szolgáltat. Ez a megközelítés lehetővé teszi, hogy a kliens oldalak teljesen független életet éljenek, miközben ugyanazokat az adatokat, autentikációs mechanizmust és üzleti szabályokat alkalmazzák. Az architektúra tehát klasszikus kliens-szerver modellt követ, azzal a kiegészítéssel, hogy két teljesen különböző típusú kliens is ugyanahhoz a backendhez csatlakozik.

A backend alkalmazás Express.js keretrendszeren alapul, és a következő főbb feladatokat látja el: felhasználói hitelesítés (JWT), projektek, feladatok és alfeladatok kezelése, sprintek és napló bejegyzések tárolása, XP-alapú gamifikációs logika végrehajtása, valamint GitHub API-val való integráció. Az adatok MongoDB adatbázisban tárolódnak, amelyhez a Mongoose ODM biztosít séma-alapú hozzáférést.

A webes frontend Vite segítségével épített React alkalmazás, amely Tailwind CSS-t használ a stílusok kialakítására. Az állapotkezelés React Context API-n és `useReducer` hookson alapul, amelyek Redux-szerű, de külső könyvtártól független megoldást nyújtanak. A mobil alkalmazás Expo keretrendszerre épített React Native projekt, amely ugyanolyan üzleti logikát követ, mint a webes változat, azonban natív mobil UI komponenseket alkalmaz.

## 3.2. Technológiai stack

A fejlesztés során a technológiai döntések több szempontot is figyelembe vettek: a fejlesztői ökoszisztéma érettségét, a közösségi támogatottságot, a típusbiztonságot, va-




3.1. ábra. Az Acxor rendszer magas szintű architektúrája

lamint azt, hogy a választott technológiák lehetővé tegyék a gyors prototípuskészítést anélkül, hogy a kód minőségét veszélyeztetnék.

### 3.2.1. Backend

A backend Node.js és TypeScript kombinációjára épül. A TypeScript alkalmazása biztosítja a statikus típusellenőrzést, ami különösen hasznos az API végpontok bemeneti és kimeneti adatainak validálásakor. Az Express.js a de facto standard REST API keretrendszer Node.js alatt, amelynek middleware-alapú felépítése rugalmasan bővíthető. A MongoDB dokumentumorientált adatbázis jól illeszkedik a projektmenedzsment adatok természetéhez, ahol a rugalmas sémák lehetővé teszik a feladatok, alfeladatok és projektek összetett struktúráinak tárolását. A Mongoose ODM séma validációt és referencia-alapú kapcsolatokat biztosít a dokumentumok között.

A hitelesítés JSON Web Token alapon működik. Bejelentkezéskor a szerver egy 7 napos érvényességű tokent állít ki, amelyet a kliens minden API kéréshez csatol az `Authorization` fejlécben. A jelszavak bcryptjs segítségével kerülnek hashelésre, 10-es salt rounds értékkel. A védett végpontokat egy `protect` middleware védi, amely ellenőrzi a token érvényességét és azonosítja a felhasználót.

### 3.2.2. Webes frontend

A webes alkalmazás React 18-ra épül, Vite fejlesztői szerverrel és build eszközzel. A Tailwind CSS utility-first megközelítése lehetővé tette a gyors és következetes stílusok kialakítását anélkül, hogy külön CSS fájlokat kellett volna karbantartani. Az állapotkezelés teljesen egyedi megoldást alkalmaz: a `GlobalContext` tárolja az alkalmazás fő adatait (felhasználó, projektek, feladatok, alfeladatok, sprintek), míg a `ViewContext` a nézetek közötti navigációért felelős. Az axios könyvtár HTTP kérések küldésére szolgál a webes kliensben.

### 3.2.3. Mobil frontend

A mobil alkalmazás React Native alapú, Expo keretrendszerrel. Az Expo lehetővé teszi a gyors fejlesztést és az eszközön való tesztelést Expo Go segítségével. A navigáció Expo Router révén fájlrendszer-alapú, ami közelebb áll a Next.js szemléletéhez. A mobil alkalmazás a fetch API-t alkalmazza HTTP kérésekhez, és a backend ugyanazon végpontjait éri el, mint a webes kliens. Az állapotkezelés megegyezik a webes verzióéval: Context API és useReducer kombináció.

## 3.3. Karbantarthatósági szempontok a kliens architektúrában

A háromplatformos architektúra egyik legkritikusabb tervezési kérdése az volt, hogy hogyan lehet a webes React alkalmazás és a React Native mobil alkalmazás között a kódot és az üzleti logikát úgy szervezni, hogy a rendszer hosszú távon karbantartható maradjon. Ez a szempont már a tervezési fázisban meghatározó szerepet játszott, hiszen a két kliens párhuzamos fejlesztése és jövőbeli módosítása komoly konzisztencia-problémákat vethet fel, ha az architektúra nem veszi figyelembe a karbantarthatóság elveit.

### 3.3.1. Közös állapotkezelési minta

A karbantarthatóság szempontjából az egyik legfontosabb döntés az volt, hogy mindkét kliensalkalmazás ugyanazt az állapotkezelési mintát alkalmazza: React Context API-t és `useReducer` hookot. Ez azt jelenti, hogy a `GlobalContext`, a `Reducer` és az `Actions` struktúrája, valamint az action típusok elnevezési konvenciói azonosak a webes és a mobil alkalmazásban egyaránt.

Ennek a döntésnek két közvetlen karbantarthatósági előnye van. Egyrészt, ha egy fejlesztő a webes kliensben elsajátítja az állapotkezelési mintát, azonnal képes a mobil kliensben is eligazodni és módosításokat végezni – nincs szükség egy teljesen eltérő paradigma megismerésére. Másrészt, ha az üzleti logika változik (például új mezők kerülnek a feladatmodellbe), az action típusok és a reducer ágak mindkét kliensben ugyanolyan struktúrájú módosítást igényelnek, ami csökkenti az elcsúszás (divergencia) kockázatát.

### 3.3.2. Kódduplikáció és annak tudatos kezelése

A két kliens között azonban szükségszerűen van kódduplikáció: az `Actions.ts` fájlban található aszinkron action creator függvények lényegében ugyanazt az API-hívási logikát valósítják meg mindkét platformon, csupán a HTTP könyvtár tér el (axios a weben, fetch a mobilon). Ez a duplikáció tudatos kompromisszum: az alternatíva egy közös, megosztott csomag (shared package) kialakítása lett volna, ami monorepo struktúrát igényelt volna (például Nx vagy Turborepo segítségével).

A monorepo megközelítés karbantarthatósági előnye vitathatatlan: a közös logikát egyetlen helyen kellene módosítani, és a változás automatikusan érvényre jutna mindkét kliensben. Ugyanakkor a monorepo bevezetése jelentős konfigurációs többletmunkával jár, és egy egyszemélyes fejlesztői projekt keretein belül az ebből fakadó komplexitás nem arányos az előnyökkel. Ezért a jelen implementációban a két kliens egymástól

független repositoryként él, a duplikált logika pedig manuálisan szinkronizált. Ez egy tudatosan vállalt karbantarthatósági adósság (technical debt), amelynek felszámolása a jövőbeli fejlesztések egyik iránya lehet.

### 3.3.3. A közös backend mint karbantarthatósági horgony

A két kliens közötti konzisztencia legfőbb biztosítéka a közös backend API. Mivel mindkét kliens ugyanazokat a végpontokat hívja, ugyanazokat a JSON struktúrákat kapja válaszul, és ugyanazokat a hibaüzeneteket kezeli, a backend az egyetlen igazságforrás (single source of truth) a rendszerben. Ha az adatmodell változik – például egy új mező kerül a `Task` dokumentumba –, a backend egyetlen módosítása elegendő ahhoz, hogy mindkét kliens az új adatot megkapja. A klienseknek csak a megjelenítési réteget kell frissíteniük.

Ez az elv különösen jól érvényesül a TypeScript-alapú fejlesztés kontextusában: a backend Mongoose modelljeihez tartozó TypeScript interfészek (`ITask`, `IProject` stb.) elméletileg exportálhatók és a kliensekben importálhatók lennének egy megosztott típuscsomagból. A jelenlegi implementációban ezek a típusok mindkét kliensben kézzel vannak definiálva, ami azt jelenti, hogy egy adatmodell-változáskor mindhárom helyen (backend, webes kliens, mobil kliens) szükséges a frissítés. Ez az API contract szinkronizálási kihívás szintén a jövőbeli fejlesztések egyik prioritása.

### 3.3.4. Tesztelhetőség mint karbantarthatósági alap

A karbantarthatóság és a tesztelhetőség szorosan összefügg: egy jól tesztelt rendszerben a módosítások hatása kontrollálható, a regressziók gyorsan felismerhetők. Az Acxor architektúrájában a karbantarthatóságot szolgáló tesztelési döntések közé tartozik a backend service réteg (xpService, logService, githubService) elkülönítése a route handlererektől. Ez a szétválasztás lehetővé teszi, hogy az üzleti logika önállóan, a HTTP réteg nélkül tesztelhető legyen, ami egyszerre növeli a tesztek megbízhatóságát és csökkenti a tesztek futási idejét.

A frontend oldalon a Context API és useReducer pattern szintén a tesztelhetőséget szolgálja: a reducer tiszta függvény, amelynek teszteléséhez nincs szükség se böngészőre, se mockolt hálózati rétegre. Ez azt jelenti, hogy az állapotkezelési logika változásakor a regressziók azonnal felismerhetők anélkül, hogy teljes komponens-renderelést kellene végezni.

### 3.3.5. Navigációs architektúra és karbantarthatóság

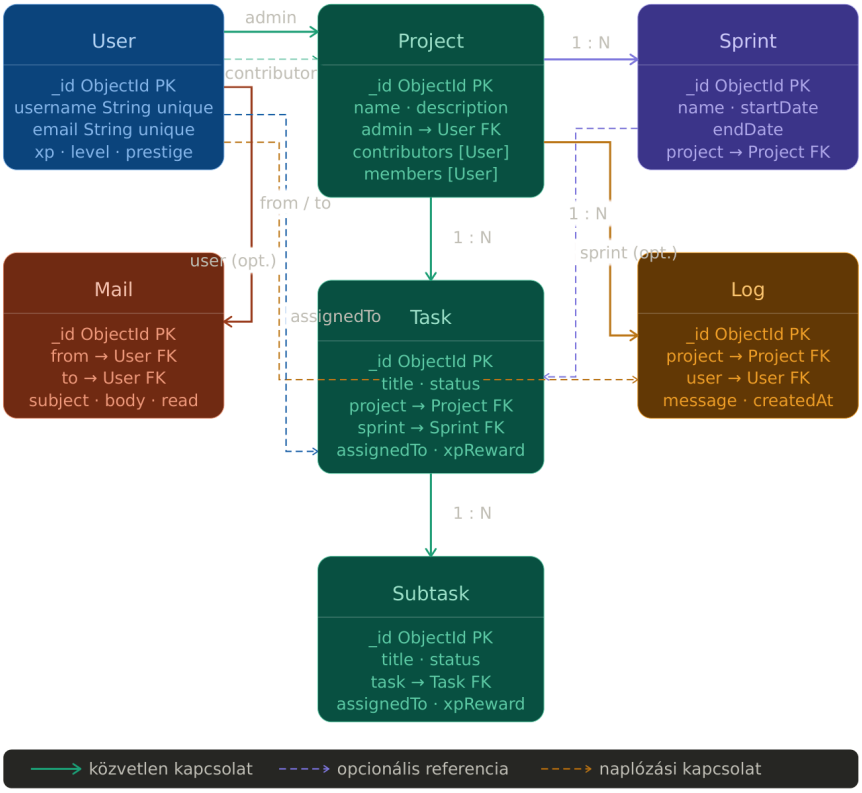
A webes és a mobil alkalmazás navigációs megközelítése szándékosan különbözik egymástól, és ez a különbség szintén karbantarthatósági szempontból indokolt. A webes alkalmazás állapot-alapú, `ViewContext`-vezérelt navigációt alkalmaz, míg a mobil alkalmazás Expo Router fájlrendszer-alapú navigációt használ. Bár ez a különbség kód-duplikációt okoz a navigációs logikában, mindkét megközelítés a saját platformján a legtermészetesebb és legkarbantarthatóbb megoldás: a webes SPA kontextusában az állapot-alapú nézetes navigáció egyszerűbb és kevesebb függőséget igényel, míg a mobilon a fájlrendszer-alapú routing a React Native ökoszisztéma bevett, jól dokumentált konvenciója, amelynek követése csökkenti a jövőbeli fejlesztők betanulási idejét.

## 3.4. Adatbázis-tervezés

Az adatbázis tervezésekor a legfontosabb szempontok a normalizált, de praktikusan használható séma kialakítása volt. A MongoDB dokumentumorientált természete lehetővé tette a referencia-alapú kapcsolatok (ObjectId hivatkozások) és a beágyazott dokumentumok vegyítését. A rendszer összesen hét Mongoose modellt alkalmaz.

### 3.4.1. Adatmodellek

A `User` modell tartalmazza a felhasználói adatokat, köztük a gamifikációhoz szükséges `xp`, `level` és `prestige` mezőket. A `Project` dokumentum tárolja az adminisztrátor, a közreműködők és a megtekintők referenciáit, ezáltal biztosítva az alapszintű hozzáférésvezérlést. A `Task` modell egy projektre és opcionálisan egy sprintre mutat, emellett tartalmaz `xpReward`, `deadline` és `assignedTo` mezőket. A `Subtask` modell szülő feladathoz kapcsolódik, és hasonló szerkezetet követ. A `Sprint` modell projekthez tartozik, és dátumtartománnyal rendelkezik. A `Log` modell eseménynaplózásra szolgál, projekt-szinten. A `Mail` modell a felhasználók közötti üzenetküldést teszi lehetővé.




3.2. ábra. Az Acxor rendszer Osztálydiagramja

Az alábbi táblázat összefoglalja a főbb modellek legfontosabb mezőit:

### 3.4.2. XP és szintlépési rendszer tervezése

A gamifikációs rendszer tervezésekor fontos szempont volt, hogy a szintlépési formula megfelelően skálázódjon: az alacsonyabb szinteken legyen könnyen teljesíthető, de

| Modell | Mező | Leírás |
|---|---|---|
| User | username, email, password | Alapadatok, egyedi azonosítók |
| | xp, level, prestige | Gamifikációs állapot |
| | role, verified, avatar | Jogosultság és profil |
| Project | name, description | Projekt azonosítása |
| | admin, contributors, viewers, members | Hozzáférésvezérlés (ObjectId ref.) |
| | createdBy | Tulajdonos |
| Task | title, status | Feladat neve és állapota |
| | project, sprint | Szülő referenciák |
| | assignedTo, deadline, xpReward | Hozzárendelés és jutalom |
| Subtask | title, status | Alfeladat neve és állapota |
| | task, assignedTo, xpReward | Szülő feladat és jutalom |
| Sprint | name, project, startDate, endDate | Sprint adatok |
| Log | project, message, user | Naplóbejegyzés |
| Mail | from, to, subject, body, read | Üzenet adatok |

3.1. táblázat. Az adatmodellek főbb mezői

magasabb szinteken és presztízs értéknél egyre több munkát igényeljen. A választott formula:

$$\text{XP szükséges} = 20 + (\text{következő szint} \times 10) + (\text{prestige} \times \text{szint} \times 10) \tag{3.1}$$

Ez a formula biztosítja, hogy az első szintlépés viszonylag kevés XP-vel elérhető, ugyanakkor a prestige rendszer exponálisan növeli a szintenként szükséges XP mennyiséget. Az `addXP` service függvény a szintlépési logikát egy `while` ciklusban kezeli, hogy egyszerre több szinten is lehetséges legyen lépni, ha elég XP gyűlik össze. A jutalmak a következők: befejezett feladat esetén alapértelmezetten 5 XP, befejezett alfeladat esetén 3 XP jár a felelős felhasználónak.

## 3.5. API tervezés

Az API RESTful elveket követ: az erőforrások azonosítása URL-eken keresztül történik, az HTTP metódusok (GET, POST, PUT, DELETE) jelölik a szándékot, a válaszok JSON formátumúak, és a megfelelő HTTP státuszkódok kerülnek visszaadásra. Az összes érzékeny végponthoz Bearer token alapú hitelesítés szükséges.

### 3.5.1. Végpont-struktúra

Az API végpontjai a következő fő útvonalcsoportokra oszthatók:

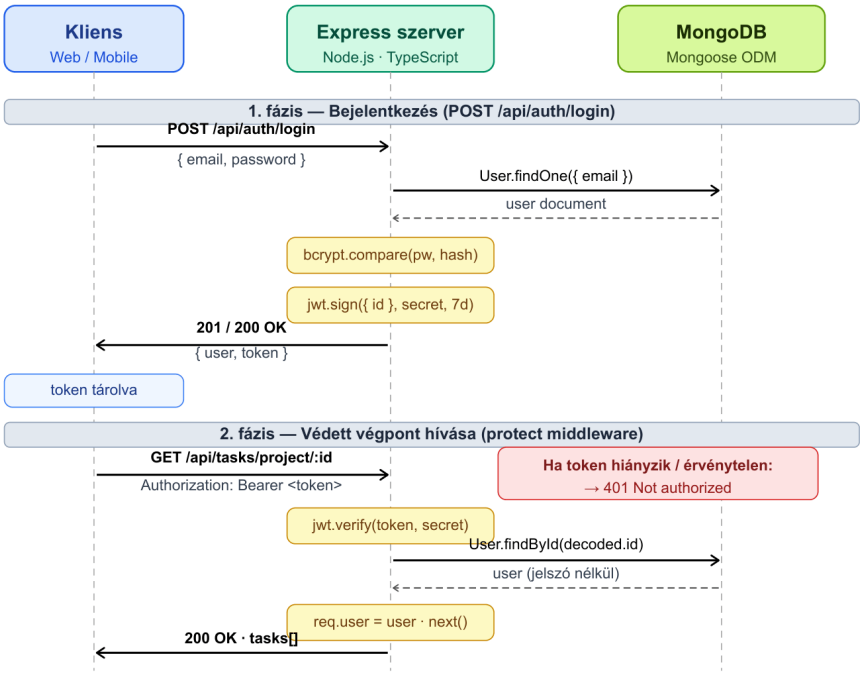
- `/api/auth` – Regisztráció és bejelentkezés
- `/api/projects` – Projektek CRUD műveletek

- `/api/tasks` – Feladatok kezelése, befejezés XP-vel
- `/api/subtasks` – Alfeladatok kezelése, befejezés XP-vel
- `/api/sprints` – Sprintek és feladat hozzárendelés
- `/api/logs` – Projektek aktivitásnaplója
- `/api/users` – Saját profil lekérése és módosítása
- `/api/mails` – Belső üzenetrendszer
- `/api/github` – GitHub integrációs végpontok

3.3. ábra. Az Acxor REST API dokumentációja (Swagger UI)

### 3.5.2. Hitelesítési folyamat

A hitelesítés kétlépéses folyamat: regisztrációkor a szerver bcrypt segítségével hasheli a jelszót, majd JWT tokent állít ki. Bejelentkezéskor az elküldött jelszót összehasonlítja a tárolt hashsel, és sikeres egyezés esetén új tokent bocsát ki. A token tartalmazza a felhasználó azonosítóját (`id` mező), 7 napos érvényességgel. A védett middleware a tokent dekódolja, megkeresi az adatbázisban a felhasználót, és ha megtalálja, csatolja a `req.user` objektumhoz a további feldolgozáshoz.




3.4. ábra. A JWT alapú hitelesítési folyamat szekvenciadiagramja

### 3.5.3. Feladat-befejezési folyamat

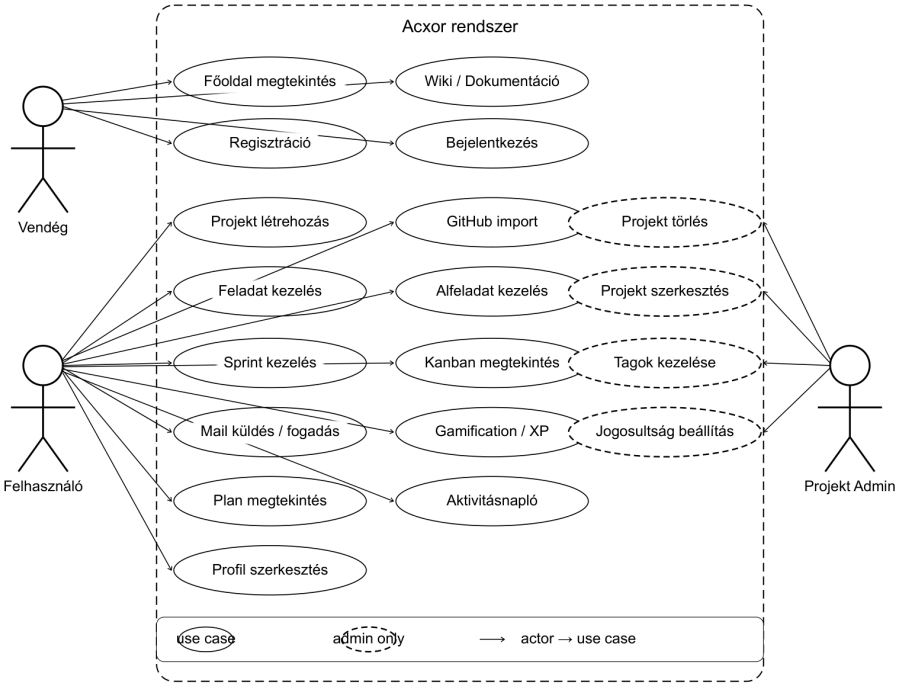
A feladatok befejezése nem az általános UPDATE végponton keresztül történik, hanem egy dedikált `PUT /api/tasks/:id/complete` végponton. Ez a döntés azért született, mert a befejezéshez kapcsolódó logika (XP jóváírás, naplóbejegyzés írása) komplex mellékhatásokat von maga után, amelyeket el kell különíteni az egyszerű adatmódosítástól. A befejezési végpont ellenőrzi, hogy a feladat korábban már el volt-e végezve (`alreadyDone` jelző), és csak az első befejezéskor ír jóvá XP-t. Hasonló logika érvényes az alfeladatokra is.

## 3.6. Use case tervezés

A rendszer főbb használati esetei a következő szereplők köré szerveződnek: vendég felhasználó (nem bejelentkezett), alapfelhasználó (bejelentkezett), és projekt adminisztrátor (a projekt létrehozója). A vendég felhasználók csak a főoldalt és a wiki oldalt tekinthetik meg. Az alapfelhasználók projekteket hozhatnak létre és kezelhetnek, feladatokat adhatnak hozzá és zárhatnak le, belső üzeneteket küldhetnek, és követhetik a gamifikációs előrehaladásukat. A projekt adminisztrátor emellett módosíthatja és törölheti a projektet, és kezelheti a közreműködők listáját.

## 3.7. Felhasználói felület tervezése

A felhasználói felület tervezésekor a vezérelv az volt, hogy a rendszer egyszerre legyen professzionális és könnyen kezelhető. Az esztétikai döntések a kék szín különböző árnyalataira épülnek, amelyek egységes vizuális identitást biztosítanak a három platform között.




3.5. ábra. Az Acxor rendszer use case diagramja

### 3.7.1. Webes alkalmazás elrendezése

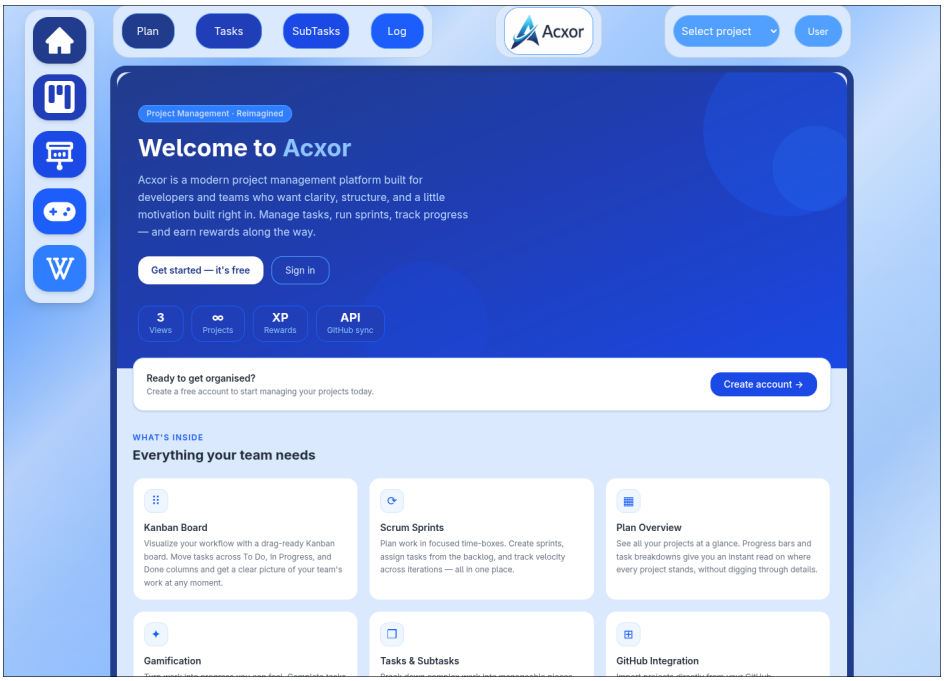
A webes felület három fő területre osztható: oldalsó navigációs sáv (Sidebar), felső eszköztár (Topbar) és a fő tartalmi terület. Az oldalsó sáv öt ikont tartalmaz: főoldal, Kanban, Scrum, gamifikáció és wiki. A felső eszköztár tartalmaz nézetek közötti váltógombokat (Plan, Tasks, Subtasks, Log), egy projekt-választó legördülő menüt és egy felhasználói menüt. A tartalmi terület minden nézetben egységes „hero" fejléccel kezdődik, amely gradiens háttérrel, projekt nevével és releváns statisztikákkal indul.

### 3.7.2. Mobil alkalmazás elrendezése

A mobil alkalmazásban a navigáció egy vízszintesen görgethető tab sávon keresztül valósul meg, amely tíz lapot tartalmaz. Minden egyes lap egy-egy főnézetet jelenít meg. A tartalom ScrollView-ban jelenik meg, SafeAreaView burkolóval. A hero fejlécek megmaradtak a webes verzióhoz képest, de a komponensek natív React Native komponensek (View, Text, TouchableOpacity, StyleSheet). A projektválasztás egy modal alapú picker segítségével történik.

### 3.7.3. Nézetek és navigáció

A webes alkalmazásban a navigáció egy egyedi `ViewContext` alapú rendszeren keresztül valósul meg, amely a hagyományos URL-alapú útválasztás helyett állapot-alapú nézetes rendszert alkalmaz. Ez azt jelenti, hogy az összes nézet egyetlen oldalon belül jelenik meg, és a `SET_VIEW` action típusú dispatch hívás váltja azokat. Védett nézetek esetén (pl. Kanban, Tasks, Scrum) a rendszer bejelentkezési képernyőre irányítja a nem azonosított felhasználókat.




3.6. ábra. A webes alkalmazás főoldalának elrendezése

A mobil alkalmazásban az Expo Router fájlrendszer-alapú navigációt alkalmaz: a `(tabs)` mappában lévő fájlok automatikusan tab lapokként kerülnek regisztrálásra. Az önálló képernyők (Sign, Create Project) Stack navigáció részeként jelennek meg.

## 3.8. Állapotkezelés tervezése

Az állapotkezelés mindkét kliensalkalmazásban React Context API-ra és useReducer hookra épül. Ez a megközelítés a Redux tároló-action-reducer mintáját követi, de anélkül, hogy külső könyvtár függőséget vonna be. A webes alkalmazásban két Context létezik párhuzamosan: a `GlobalContext` és a `ViewContext`.

A `GlobalContext` a következő állapotmezőket tartalmazza: `user` (bejelentkezett felhasználó adatai), `token` (JWT token), `projects` (projektek listája), `tasks` (aktuális projekt feladatai), `subtasks` (feladatokhoz tartozó alfeladatok), `sprints` (sprintek), `selectedProject` (kiválasztott projekt), `loading` és `error` (kérés állapot jelzők).

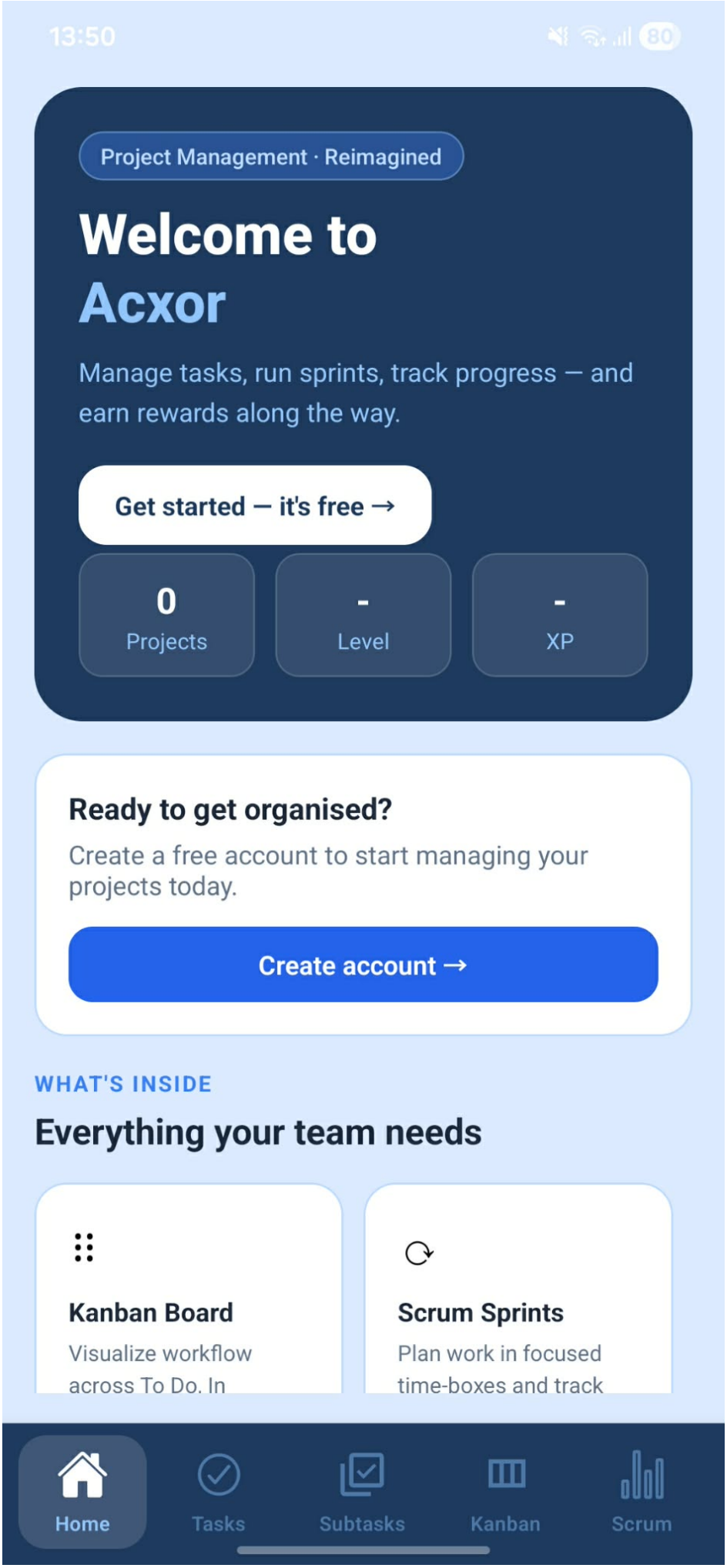
Az `APPEND_SUBTASKS` action típus különösen figyelemre méltó: mivel több feladat alfeladatait töltjük be párhuzamosan, a hagyományos `SET_SUBTASKS` felülírná a korábbi eredményeket. Az `APPEND_SUBTASKS` ezért az adott feladathoz tartozó korábbi alfeladatokat kiszűri, majd hozzáfűzi az új eredményt, így az összes betöltött alfeladat megmarad az állapotban.

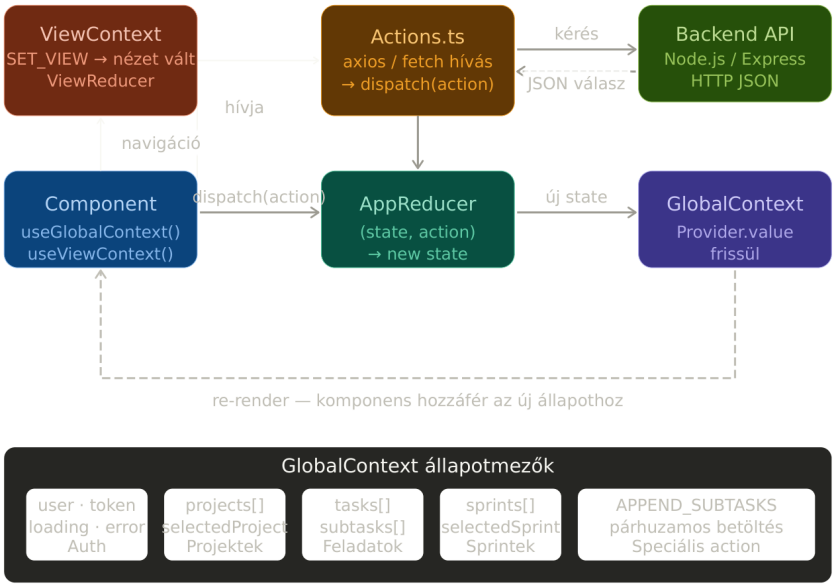
## 3.9. GitHub integráció tervezése

Az Acxor egyik kiegészítő funkciója a GitHub repozitóriumokból való projektimportálás. A tervezés során el kellett dönteni, hogy ez a funkció szerver oldalon vagy kliens oldalon menjen végbe. A szerver oldali megközelítés mellett döntöttünk, mivel a GitHub API tokent nem célszerű közvetlenül a kliensből küldeni a GitHub felé: a szerver

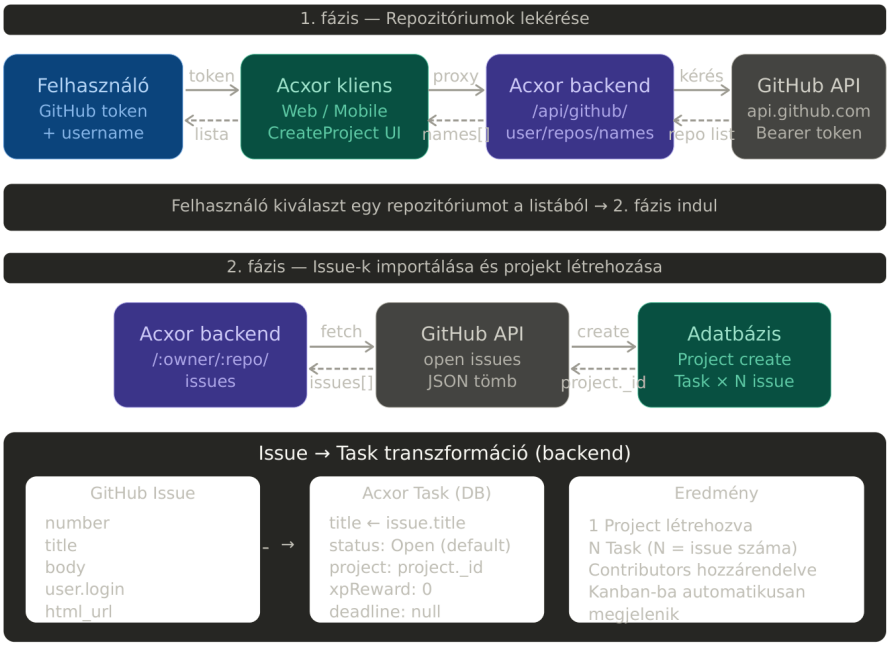
közvetítőként működik, ami lehetővé teszi a jövőbeli token validációs logika hozzáadását is.

A folyamat a következő lépésekből áll: a felhasználó megadja GitHub felhasználónevét és személyes hozzáférési tokenét, a kliens ezeket elküldi a backendbeli `/api/github/user/repos/n` végpontra, a szerver a GitHub API segítségével lekéri a repozitóriumok listáját, majd visszaküldi a neveket. A felhasználó kiválaszt egy repozitóriumot, amelynek issue-it a backend lekéri a `/api/github/:owner/:repo/issues` végponton keresztül. Az issuesek feladatokká alakulnak, és egy projektlétrehozási hívással együtt kerülnek az adatbázisba.


13:50
80
Project Management · Reimagined
Welcome to
Acxor
Manage tasks, run sprints, track progress — and earn rewards along the way.
Get started — it's free →
0
Projects
-
Level
-
XP
Ready to get organised?
Create a free account to start managing your projects today.
Create account →
WHAT'S INSIDE
Everything your team needs
Kanban Board
Visualize workflow across To Do, In
Scrum Sprints
Plan work in focused time-boxes and track
Home
Tasks
Subtasks
Kanban
Scrum




3.8. ábra. A Context API alapú állapotkezelés folyamata




3.9. ábra. A GitHub integráció adatfolyam-ábrája

# 4. fejezet

# Implementáció

Az implementációs fejezet részletesen bemutatja, hogyan valósultak meg a tervezési fázisban lefektetett elvek a konkrét kód szintjén. A fejezet a backend szolgáltatásoktól és adatmodellektől indul, majd a frontend állapotkezelésen és komponensstruktúrán át eljut a tesztelésig. Ahol indokolt, kódrészletek is szerepelnek az érintett mechanizmusok szemléltetéséhez.

## 4.1. Backend implementáció

### 4.1.1. Projekt struktúra

A backend kódja az alábbi könyvtárszerkezetbe szerveződik:

Listing 4.1. Backend könyvtárstruktúra

```
backend/
  app.ts              # Express app konfiguráció, route regisztráció
  server.ts           # Szerver indítás, DB kapcsolat
  middleware/
    auth.ts           # JWT védő middleware
  models/             # Mongoose modellek (User, Project, Task, ...)
  routes/             # Express route handlerek
  controllers/        # Vezérlők (Task, GitHub)
  services/
    xpService.ts      # XP és szintlépési logika
    logService.ts     # Naplóbejegyzés írás
    githubService.ts  # GitHub API hívások
  config/
    db.ts             # MongoDB kapcsolat
  __tests__/          # Jest + Supertest integrációs tesztek
```

Az `app.ts` fájl az Express alkalmazást konfigurálja: engedélyezi a CORS-t, beállítja a JSON body parsert, és regisztrálja az összes route-ot a megfelelő előtag alá. Maga az Express app objektum exportálva van, és a `server.ts` indítja el a figyelő portot – ez a szétválasztás lehetővé teszi, hogy a tesztek közvetlenül az alkalmazáson fussanak anélkül, hogy valóban portot foglalnának.

### 4.1.2. Mongoose modellek

A modellek TypeScript interfészeket definiálnak (`IUser`, `IProject` stb.), amelyek lehetővé teszik a típusbiztos hozzáférést a dokumentum mezőkhöz. Az alábbiakban a `User` modell megvalósításának főbb részei láthatók:

Listing 4.2. User modell (models/User.ts)

```
const UserSchema = new Schema<IUser>(
  {
    username: { type: String, required: true, unique: true },
    email:    { type: String, required: true, unique: true },
    password: { type: String, required: true },
    xp:       { type: Number, default: 0 },
    level:    { type: Number, default: 1 },
    prestige: { type: Number, default: 0 },
    role:     { type: String, default: 'user' },
  },
  { timestamps: true }
);
```

A `timestamps: true` beállítás automatikusan kezeli a `createdAt` és `updatedAt` mezőket. A `Project` modellben a kapcsolódó felhasználókat `Types.ObjectId` referenciákon keresztül tároljuk, amelyek a Mongoose `populate()` metódusával tölthetők ki tényleges adatokkal a lekérdezésekkor.

### 4.1.3. Hitelesítés implementációja

A regisztrációs route (`POST /api/auth/register`) ellenőrzi az e-mail cím egyediségét, hasheli a jelszót, létrehozza a felhasználót, majd JWT tokent állít ki:

Listing 4.3. Regisztrációs végpont (routes/auth.ts)

```
router.post('/register', async (req, res) => {
  const { username, email, password } = req.body;
  const existingUser = await User.findOne({ email });
  if (existingUser)
    return res.status(400).json({ message: 'User already exists' });

  const hashedPassword = await bcrypt.hash(password, 10);
  const user = await User.create({ username, email,
                                   password: hashedPassword });

  const token = jwt.sign({ id: user._id },
    process.env.JWT_SECRET as string, { expiresIn: '7d' });

  res.status(201).json({ user, token });
});
```

A `protect` middleware dekódolja a Bearer tokent, megkeresi a felhasználót és csatolja a `req.user` mezőhöz. Ha a token érvénytelen vagy hiányzik, 401-es státuszkóddal tér vissza.

### 4.1.4. XP és gamifikációs szolgáltatás

Az `xpService.ts` felelős az XP jóváírásáért és a szintlépési logikáért. A függvény egy `while` ciklusban ellenőrzi, hogy a felhasználó XP-je eléri-e a következő szinthez szükséges küszöbértéket, és ha igen, csökkenti az XP-t a küszöb értékével és növeli a szintet:

Listing 4.4. XP jóváírási logika (services/xpService.ts)

```
export const addXP = async (userId: string, xp: number) => {
  const user = await User.findById(userId);
  if (!user) throw new Error('User not found');
  user.xp += xp;

  const xpPerLevel = 100;
  while (user.xp >= user.level * xpPerLevel) {
    user.xp -= user.level * xpPerLevel;
    user.level += 1;
  }

  await user.save();
  return user;
};
```

Ez az implementáció kezeli azt az esetet is, amikor egyetlen feladat befejezésével a felhasználó egyszerre több szintet lép, mivel a `while` ciklus addig fut, amíg az XP az aktuális küszöb alatt nem kerül.

### 4.1.5. Naplózási szolgáltatás

A `logService.ts` minden lényeges eseményt (feladat létrehozása, törlése, befejezése, sprint kezelés stb.) egy `Log` dokumentumként ment az adatbázisba. Fontos implementációs döntés, hogy a `logEvent` függvény hibáját soha nem engedi továbbugorni a hívó kódba: egy `try-catch` blokk belül a naplózási hiba csak konzolra kerül, így egyetlen rossz naplóbejegyzés sem törhet meg egy egyébként sikeres API hívást.

### 4.1.6. Projektlétrehozás TaskGroup-okkal

A projektlétrehozási route megvalósítása összetett, mert egyszerre kell kezelni a projekt dokumentum létrehozását, a felhasználói referenciák feloldását felhasználóneveknél, és a taskGroup struktúrában érkező feladatok és alfeladatok tömegesen létrehozását. A route végigiterál a `taskGroups` tömbön, minden feladathoz létrehoz egy `Task` dokumentumot (a csoport határidejével), majd az alfeladatokhoz `Subtask` dokumentumokat kapcsol hozzá.

Listing 4.5. TaskGroup feldolgozás projektlétrehozáskor

```
for (const group of taskGroups) {
  if (!group.tasks) continue;
  for (const taskData of group.tasks) {
    const task = await Task.create({
      title: taskData.description,
      project: project._id,
      deadline: group.deadline,
```

```
    });
    if (taskData.subtasks) {
      for (const sub of taskData.subtasks) {
        await Subtask.create({ title: sub, task: task._id });
      }
    }
  }
}
```

## 4.2. Webes frontend implementáció

### 4.2.1. Könyvtárstruktúra

A webes frontend a következő főbb könyvtárakba szerveződik:

Listing 4.6. Webes frontend könyvtárstruktúra

```
frontend/
  src/
    App.tsx                # Fő app, nézet routing
    main.tsx               # React DOM render, Providerek
    Context/
      GlobalContext.tsx    # Globális állapot Context
      ViewContext.tsx      # Nézet-váltás Context
      Reducer.tsx          # AppReducer (GlobalContext)
      ViewReducer.tsx      # ViewReducer
      Actions.ts           # Aszinkron action függvények
    components/
      Sidebar/             # Navigációs ikonok
      Topbar/              # Felső eszköztár komponensei
      Views/               # Nézetek (Kanban, Tasks, stb.)
      Sign/                # Bejelentkezés / regisztráció
      CreateProject/       # Projektlétrehozás varázsló
      Mails/               # Belső levelezés
      UserMenu/            # Profil oldal
    test/                  # Vitest + RTL unit tesztek
```

### 4.2.2. Context API és állapotkezelés

A `GlobalContext` a `useReducer` hook segítségével kezel egy komplex állapotobjektumot. Az `AppReducer` minden lehetséges action típusra tartalmaz egy ágat, és mindig új állapotobjektumot ad vissza (immutable update). A `Actions.ts` fájl aszinkron action creator függvényeket tartalmaz, amelyek axios segítségével hívják az API-t, és dispatch hívásokkal frissítik a globális állapotot.

Az alábbi kódrészlet a feladatlekérési action megvalósítását mutatja be:

Listing 4.7. fetchTasks action (Context/Actions.ts)

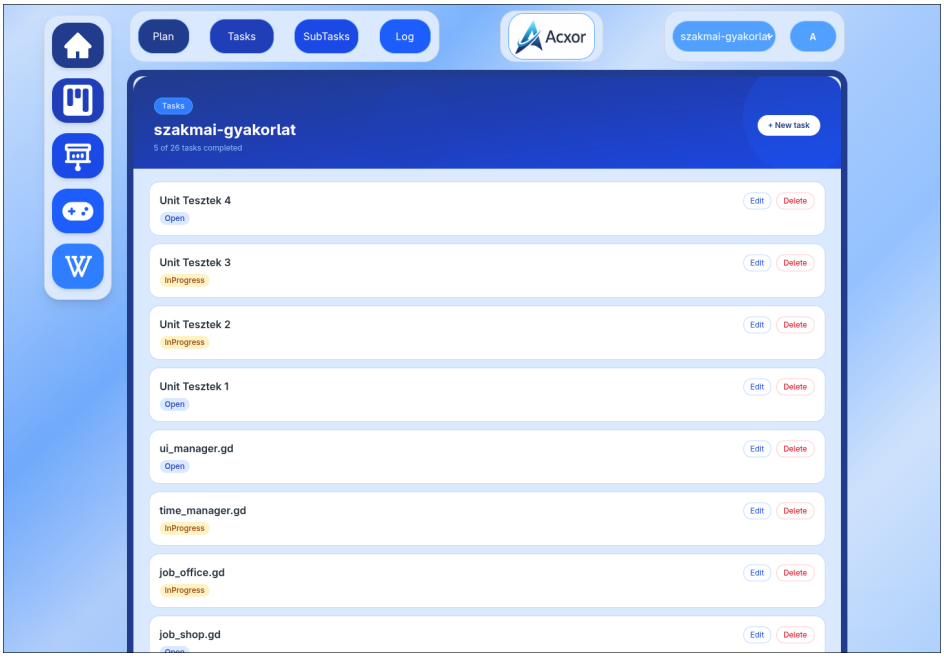
```
export const fetchTasks = async (
  dispatch: Dispatch<any>,
```

```
  token: string,
  projectId: string,
) => {
  try {
    const res = await axios.get(
      `${API_URL}/tasks/project/${projectId}`,
      { headers: { Authorization: `Bearer ${token}` } },
    );
    dispatch({ type: 'SET_TASKS', payload: res.data });
  } catch (err: any) {
    dispatch({
      type: 'SET_ERROR',
      payload: err.response?.data?.message || err.message,
    });
  }
};
```

### 4.2.3. Nézetek implementációja

Minden nézet-komponens (`TasksView`, `KanbanView`, `ScrumView` stb.) ugyanolyan struktúrát követ: a `useGlobalContext` hookkal hozzáfér az állapothoz, a `useEffect` hookkal a megfelelő adatokat tölti be (projekt-váltásra reagálva), és JSX-ben rendereli a tartalmat.

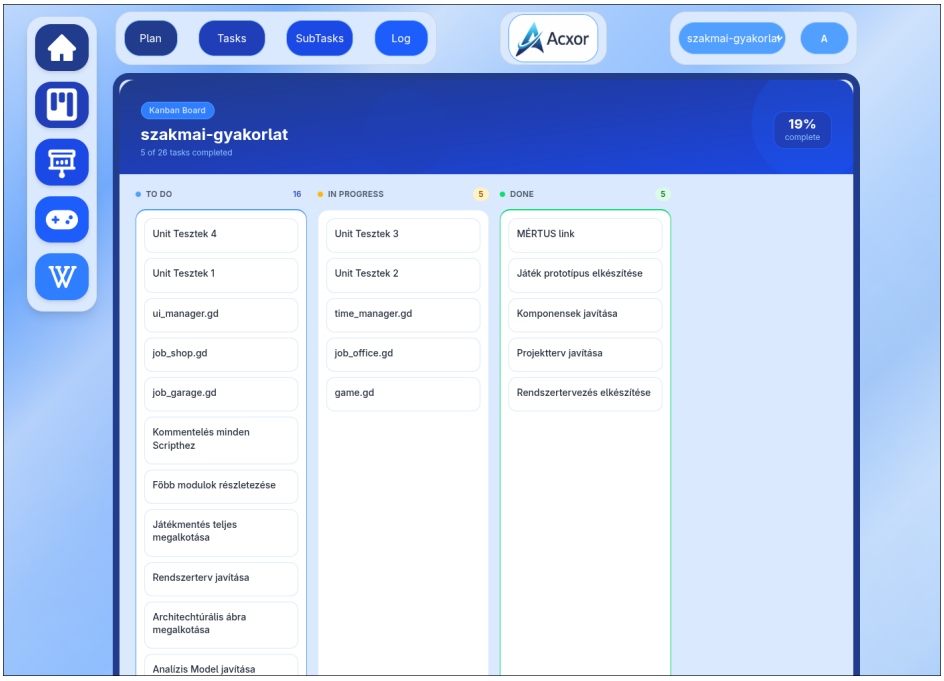
A `TasksView` komponens például az edíciós állapotot lokálisan, a `useState` hook segítségével kezeli: az `editingTask` állapot tartalmazza az éppen szerkesztett feladatot, és a kártya inline szerkesztési formát jelenít meg. A törlési és befejezési műveletek megfelelő API végpontokat hívnak, és a reducer az állapotot szinkron frissíti a szerver válasza alapján.




4.1. ábra. A TasksView komponens megjelenése

### 4.2.4. Kanban nézet

A `KanbanView` három oszlopban jeleníti meg a feladatokat (`Open`, `InProgress`, `Done`). Az oszlopok dinamikusan kiszámított adatokat kapnak: minden oszlop szűri a globális `tasks` tömbből a saját státuszához tartozó elemeket. Az oszlopok fejlécein megjelenik a benne lévő feladatok száma, az egyes kártyákon látható a felelős felhasználó és a határidő.



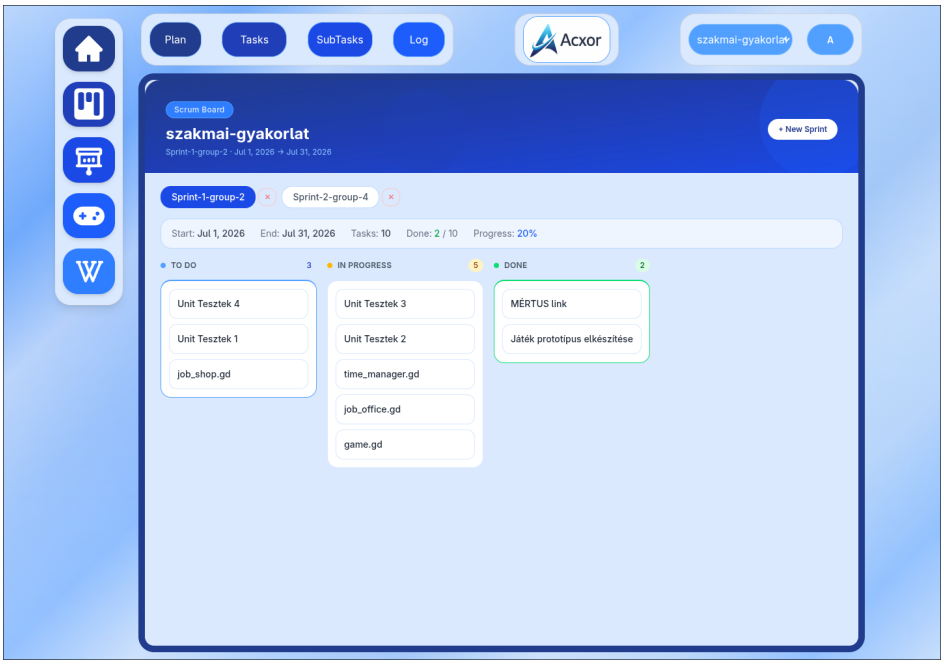

4.2. ábra. A KanbanView komponens megjelenése

### 4.2.5. Scrum nézet

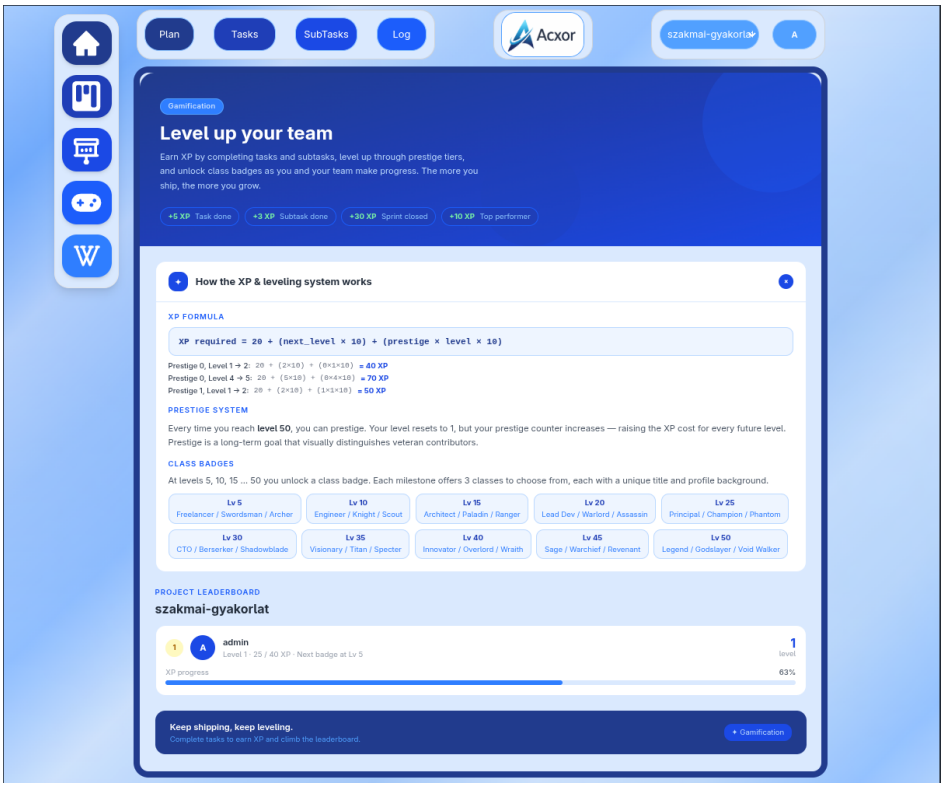
A `ScrumView` a legösszetettebb nézetek egyike: egyszerre kezeli a sprint kiválasztást, sprint létrehozást, feladatok sprinthez rendelését, és a Kanban-szerű sprint boardot. A sprinthez nem rendelt feladatok a „Backlog" szekcióban jelennek meg. A sprint kiválasztó gombok tetején az aktív sprint (amelynek dátumtartománya lefedi az aktuális dátumot) egy zöld „active" badget kap. A sprint törlése után az érintett feladatok automatikusan visszakerülnek a backlogba (a backend kezeli ezt a konzisztenciát).

### 4.2.6. Gamifikáció nézet

A `GamificationView` egy ranglistát jelenít meg a projekt tagjai számára. A felhasználókat XP és szint szerint rendezi, az első helyen lévő felhasználó kártyája arany szegéllyel kiemelve jelenik meg. Minden kártyán látható a felhasználó szintje, XP értéke, a következő szinthez szükséges XP, az XP progress bar és a megszerzett badge szintek. A gamifikációs rendszer magyarázatát egy kinyitható accordion panel tartalmazza, amelyben a szintlépési formula, a prestige rendszer és a class badge-ek kerülnek bemutatásra.




4.3. ábra. A ScrumView komponens megjelenése




4.4. ábra. A GamificationView komponens megjelenése

### 4.2.7. Mail rendszer

A levelezési rendszer (`Mails` komponens) háromrészes elrendezést alkalmaz: egy hero fejléc, egy bal oldali levéllista panel (inbox / sent tab váltóval) és egy jobb oldali olvasó/szerkesztő panel. Az olvasási műveletnél a leveleket automatikusan olvasottnak jelöli a rendszer egy `PUT` kéréssel, és az olvasatlan levelek száma a hero fejlécben és

az inbox tab feliratán is megjelenik. Az összeállítási nézet inline jelenik meg a jobb panelben, nem modálban.

## 4.3. Mobil alkalmazás implementáció

### 4.3.1. Navigáció

A mobil alkalmazás navigációja Expo Router segítségével valósul meg. A `(tabs)` csoportban tíz tab lap él: index (főoldal), tasks, subtasks, kanban, scrum, plan, gamification, mails, log, account. Az önálló képernyők (sign, create-project) a Stack navigátor részeként érhetők el. A tab bar egyedi komponensként lett megvalósítva (`CustomTabBar`), mivel az alapértelmezett Expo tab bar nem támogatta a vízszintes görgetést, amely tíz tab esetén szükségessé vált.

Listing 4.8. Egyedi tab bar implementáció (*layout.tsx*)

```
function CustomTabBar({ state, navigation }) {
  return (
    <View style={styles.tabBarWrapper}>
      <ScrollView horizontal showsHorizontalScrollIndicator={false}>
        {TABS.map((tab, index) => {
          const focused = state.index === index;
          return (
            <TouchableOpacity
              key={tab.name}
              style={[styles.tabItem,
                      focused && styles.tabItemActive]}
              onPress={() => navigation.navigate(tab.name)}
            >
              {tab.icon(focused)}
              <Text style={[styles.tabLabel,
                            focused && styles.tabLabelActive]}>
                {tab.label}
              </Text>
            </TouchableOpacity>
          );
        })}
      </ScrollView>
    </View>
  );
}
```

### 4.3.2. Projekt választó

Mivel a mobil alkalmazásban nincs topbar-szerű elem, a projektválasztás egy modal alapú `ProjectPicker` komponensen keresztül valósul meg. A komponens egy `TouchableOpacity` gombot jelenít meg az aktuális projekt nevével, és kattintásra egy alulról felcsúszó modal nyílik, amelyen az összes projekt listázva jelenik meg. A kiválasztott projekt azonnal frissíti a globális állapotot.

### 4.3.3. Alfeladatok nézet

A mobil `SubtasksScreen` egy kinyitható/becsukható struktúrát alkalmaz: minden feladatkártya fejléce koppintható, és mutatja vagy elrejti az adott feladathoz tartozó alfeladatokat. Ez a döntés szükséges volt a mobilon elérhető kisebb képernyős terület miatt. Az alfeladatok hozzáadása inline szövegbeviteli mezőn keresztül, szerkesztésük pedig egy alulról felemelkedő modálban történik.

### 4.3.4. API kommunikáció

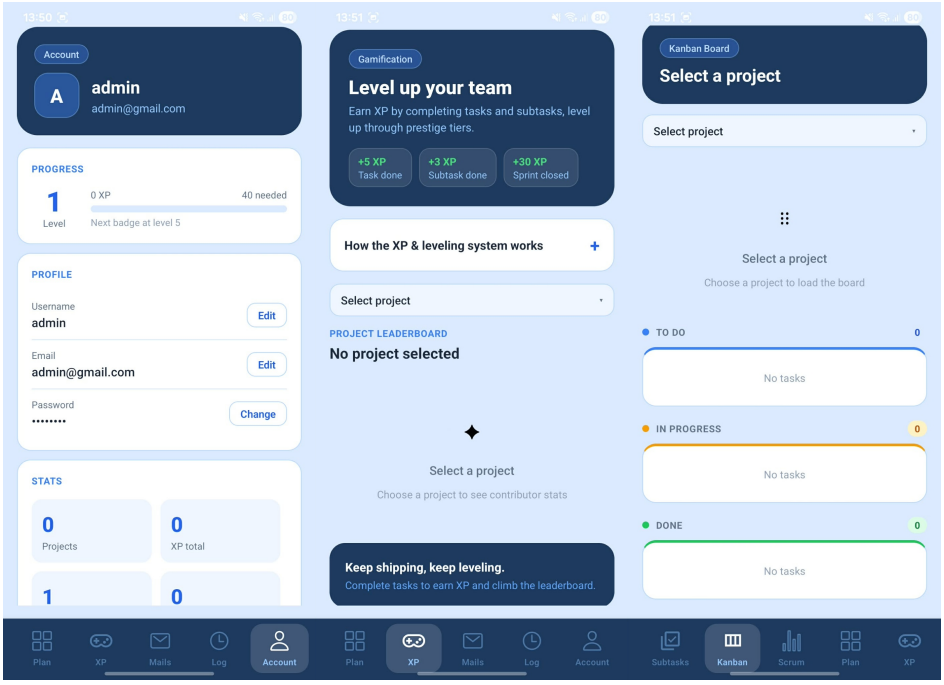
A mobil alkalmazás a webes klienstől eltérően nem axios-t, hanem a beépített `fetch` API-t használja a HTTP kérésekhez. Ez tudatos döntés volt: csökkenti a függőségek számát, és a React Native fejlesztők által is preferált megoldás. Az API URL-t a `constants/api.ts` fájl konfigurálja dinamikusan: Expo Go fejlesztői módban a `Constants.expoConfig.hostUri` alapján automatikusan meghatározza a fejlesztői gép LAN IP-jét, így fizikai eszközön is működik a fejlesztés.

Listing 4.9. Dinamikus API URL (constants/api.ts)

```
const debuggerHost =
  Constants.expoConfig?.hostUri?.split(':')[0];

const BASE = debuggerHost
  ? `http://${debuggerHost}:5000/api`
  : 'http://localhost:5000/api';

export const API_URL = BASE;
```




4.5. ábra. A mobil alkalmazás főbb nézetei

## 4.4. Tesztelés implementációja

A rendszer tesztelése két szinten valósul meg: backend oldalon Jest és Supertest segítségével írt integrációs tesztek, frontend oldalon Vitest és React Testing Library alapú unit tesztek.

### 4.4.1. Backend tesztelés

A backend tesztek in-memory MongoDB adatbázist alkalmaznak (`mongodb-memory-server`), ami azt jelenti, hogy a tesztek nem igényelnek élő adatbázis-kapcsolatot, és teljesen izoláltan futnak. A `setup.ts` fájl minden tesztfájl előtt elindítja a memóriaadatbázist, és minden egyes teszt után törli az összes kollekcióból az adatokat, hogy biztosítsa az izolált futtatást.

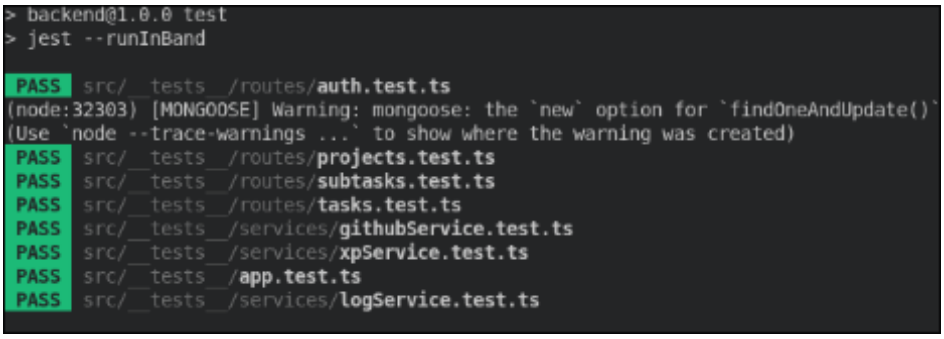
Listing 4.10. Teszt setup (__tests__/setup.ts)

```
beforeAll(async () => {
  mongoServer = await MongoMemoryServer.create();
  await mongoose.connect(mongoServer.getUri());
});

afterEach(async () => {
  const collections = mongoose.connection.collections;
  for (const key in collections)
    await collections[key].deleteMany({});
});

afterAll(async () => {
  await mongoose.disconnect();
  await mongoServer.stop();
});
```

Az integrációs tesztek a Supertest könyvtár segítségével közvetlenül az Express alkalmazásnak küldenek HTTP kéréseket, és ellenőrzik a visszakapott státuszkódot, a válasz testét és az adatbázis állapotát. A tesztek lefedik az auth, project, task, subtask és service szintű működést.




4.6. ábra. A backend tesztek futtatásának eredménye

### 4.4.2. Frontend tesztelés

A webes frontend tesztjei Vitest és React Testing Library segítségével készültek. A tesztek négy fő területet fednek le:

1. **Reducer tesztek**: Az `AppReducer` és `ViewReducer` függvények elszigetelt tesztelése, verifikálva, hogy minden action típus a várt állapotváltozást eredményezi.
2. **Context tesztek**: A `GlobalContext` és `ViewContext` tesztelése egyszerű teszt-komponenseken keresztül, ellenőrizve a dispatch-re adott re-render viselkedést.
3. **Action tesztek**: A `fetchTasks` és `addTask` action creator függvények tesztelése mockolt axios segítségével, verifikálva a helyes API hívást és dispatch sorozatot.
4. **Komponens tesztek**: A `Sign`, `CreateProject`, `TasksView` és `SubTasksView` komponensek renderelési és interakciós tesztjei, mockolt Context segítségével.

Listing 4.11. TasksView unit teszt (Vitest)

```
it('rendereli a TasksView-t', () => {
  vi.spyOn(GlobalContext, 'useGlobalContext').mockReturnValue({
    state: {
      tasks: [
        { _id: '1', title: 'Task 1', status: 'open' },
        { _id: '2', title: 'Task 2', status: 'done' },
      ],
      selectedProject: { _id: 'proj1' },
      // ... egyéb mezők
    },
    dispatch: vi.fn(),
  });

  render(<TasksView />);
  expect(screen.getByText('Tasks')).toBeInTheDocument();
  expect(screen.getByText('Task 1')).toBeInTheDocument();
  expect(screen.getByText('Task 2')).toBeInTheDocument();
});
```

```
✓ src/test/CreateProject/CreateProject.test.tsx (6 tests) 233ms
✓ src/test/Topbar/Topbar.test.tsx (3 tests) 428ms
    ✓ rendereli a topbart  376ms
✓ src/test/Sign/Sign.test.tsx (7 tests) 668ms
    ✓ rendereli a Sign komponenst alapértelmezetten login nézettel  364ms
✓ src/test/Sidebar/Sidebar.test.tsx (2 tests) 447ms
    ✓ rendereli az összes ikont (5 gomb)  392ms
✓ src/test/Views/TasksView.test.tsx (2 tests) 122ms
✓ src/test/Views/SubTasksView.test.tsx (2 tests) 124ms
✓ src/test/Context/ViewContext.test.tsx (2 tests) 77ms
✓ src/test/Context/GlobalContext.test.tsx (2 tests) 89ms
✓ src/test/Context/Reducer.test.tsx (10 tests) 13ms
✓ src/test/Context/ViewReducer.test.tsx (3 tests) 6ms
✓ src/test/Context/Actions.test.ts (8 tests) 12ms
```

4.7. ábra. A frontend tesztek futtatásának eredménye

### 4.4.3. GitHub service tesztelés

A `githubService.ts` tesztelése érdekes kihívást jelentett, mivel a GitHub API-t nem szerettük volna valóban hívni a tesztek során. Az axios mockolt verziójával (`jest.mock('axios')`) a tesztek ellenőrzik, hogy a service függvények a helyes URL-t hívják-e, a helyes headereket küldik-e el, és a válasz adatait a várt formátumra alakítják-e. A `logService.ts` tesztelésénél a `console.log` spy-olásával ellenőriztük, hogy a naplóbejegyzések a várt formátumban kerülnek kiírásra.

## 4.5. Fejlesztési kihívások és megoldások

A fejlesztés során számos praktikus kihívással kellett szembenézni, amelyek megoldása tanulságos volt.

**Alfeladat betöltés párhuzamos kérésekkel**: Az alfeladatokat feladatonként kellett betölteni, ami azt jelentette, hogy ha egy projektnek 10 feladata van, 10 párhuzamos API kérés fut le. Ha a Redux-szerű `SET_SUBTASKS` action-t használtuk volna, az egymást felülíró dispatch hívások miatt csak az utolsó kérés eredménye maradt volna meg. A megoldás az `APPEND_SUBTASKS` action bevezetése volt, amely az adott feladathoz tartozó korábbi alfeladatokat kiszűri, majd az újakkal egészíti ki az állapotot.

**Mobil eszköz és localhost**: Fizikai mobileszközön a `localhost` nem mutat a fejlesztői gépre, hanem magára az eszközre. Az Expo által elérhető `Constants.expoConfig.hostUri` mező tartalmazza a fejlesztői szerver IP-jét és portját, amelyből a gép LAN IP-je kinyerhető. Ez az automatikus konfiguráció tette lehetővé, hogy a mobil app fejlesztés közben fizikai eszközön is futtatható legyen.

**Sprint törlés és feladat konzisztencia**: Sprint törlésekor a hozzá rendelt feladatokon is el kell végezni a sprint referencia törlését, különben azok sosem kerülnek vissza a backlogba. A backend sprint törlési route-ja azonban ezt nem oldja meg automatikusan – ez egy ismert hiányosság, amelyet az összefoglalóban is megemlítünk a jövőbeli fejlesztési irányok között.

**Swagger dokumentáció hiánya**: Bár a tervezési fejezetben Swagger UI képernyőkép szerepel, a fejlesztés során külső Swagger dokumentáció nem készült automatikusan. Az API végpontok dokumentálása kézzel, `swagger-jsdoc` vagy hasonló eszközzel utólag is elvégezhető lett volna – ez szintén egy jövőbeli fejlesztési lehetőség.

# 5. fejezet

# Tesztelés

A tesztelési fejezet részletesen bemutatja az Acxor rendszer tesztelési stratégiáját, az alkalmazott eszközöket, a tesztesetek felépítését és az elért eredményeket. A tesztelés célja annak igazolása, hogy az implementált funkciók az elvárásoknak megfelelően működnek, és hogy a jövőbeli módosítások regressziót ne okozzanak a meglévő működésben.

## 5.1. Tesztelési stratégia

A rendszer tesztelése két jól elkülönített rétegben valósult meg. A backend oldalon integrációs tesztek ellenőrzik a teljes kérés-válasz ciklust, beleértve az adatbázissal való interakciót, a hitelesítési logikát és az üzleti szabályok érvényesítését. A frontend oldalon egységtesztek verifikálják az állapotkezelési logikát, az action creator függvényeket és a legfontosabb UI komponensek helyes renderelését.

A tesztelési megközelítés a következő elveket követi:

- Minden teszteset független és izolált: a tesztek között nincs közös állapot, az adatbázis tartalma minden teszteset után törlődik.
- A tesztek lehetőség szerint a rendszer külső viselkedését ellenőrzik, nem belső implementációs részleteket.
- A tesztek a fejlesztéssel párhuzamosan készültek, nem utólag.
- Ahol a külső rendszerek (GitHub API) nem hívhatók valódi tesztekben, mockolt adatokkal dolgozunk.

## 5.2. Backend tesztelés

### 5.2.1. Eszközök és konfiguráció

A backend tesztek Jest tesztelési keretrendszert és Supertest HTTP tesztkönyvtárat alkalmaznak. A Jest biztosítja a tesztfuttatót, az assertionokat és a mock funkcionalitást. A Supertest lehetővé teszi, hogy az Express alkalmazásnak valódi HTTP kéréseket küldjünk anélkül, hogy portot kellene foglalni, így a tesztek gyorsan és izoláltan futnak.

Az adatbázis szintű izoláció érdekében a `mongodb-memory-server` könyvtár egy in-memory MongoDB példányt indít el minden tesztfájl futtatásakor. Ez azt jelenti, hogy

a tesztek semmilyen körülmények között nem érintik az éles vagy fejlesztési adatbázist. A `__tests__/setup.ts` fájl konfigurálja ezt a mechanizmust, és a Jest `globalSetup` vagy `setupFilesAfterEnv` konfigurációján keresztül kerül betöltésre.

### 5.2.2. Auth tesztek

A hitelesítési tesztek a `POST /api/auth/register` és `POST /api/auth/login` végpontokat fedik le. A regisztrációs tesztek ellenőrzik:

- Sikeres regisztráció esetén 201-es státuszkód és érvényes JWT token visszaadása.
- A jelszó nem jelenik meg a válaszban (adatbiztonság).
- Duplikált e-mail esetén 400-as hiba és megfelelő üzenet visszaadása.
- A visszaadott token érvényes JWT formátumú (három pontosvesszővel elválasztott Base64 szegmens).
- Az alapértelmezett XP és level értékek helyesek (`xp: 0`, `level: 1`).

A bejelentkezési tesztek ellenőrzik a helyes adatokkal való sikeres bejelentkezést, a helytelen jelszóra adott 400-as választ, a nem létező e-mailre adott hibaüzenetet, és azt, hogy bejelentkezés válaszában sem szerepel a jelszó.

Listing 5.1. Regisztrációs teszt részlet (auth.test.ts)

```
it('a jelszó nem kerül vissza a válaszban', async () => {
  const res = await request(app)
    .post('/api/auth/register')
    .send(validUser);

  expect(res.body.user).not.toHaveProperty('password');
});

it('a visszaadott token érvényes JWT formátumú', async () => {
  const res = await request(app)
    .post('/api/auth/register')
    .send(validUser);

  const jwtPattern =
    /^[A-Za-z0-9-_]+\.[A-Za-z0-9-_]+\.[A-Za-z0-9-_]+$/;
  expect(res.body.token).toMatch(jwtPattern);
});
```

### 5.2.3. Projekt tesztek

A projekt tesztek a teljes CRUD ciklust lefedik. Minden teszteset saját felhasználót regisztrál (`registerAndLogin` segédfüggvény), így biztosítva az izolációt. A lefedett esetek:

- Projekt létrehozása helyes adatokkal (201), token nélkül (401), hiányzó névvel (400).

- A létrehozó automatikusan adminisztrátor és tag lesz.
- TaskGroup-okkal való projektlétrehozáskor a feladatok és alfeladatok létrejönnek az adatbázisban.
- Érvénytelen tokennel való kísérlet 401-et ad vissza.
- A projektek listázása csak a saját projektek tartalmazza (más felhasználóé nem jelenik meg).
- Az admin mező populálva érkezik vissza (jelszó nélkül).
- Projekt módosítása és törlése csak az adminisztrátornak engedélyezett (másnak 403).
- Törléskor a kapcsolódó feladatok és alfeladatok is törlődnek.

Listing 5.2. Projekt törlési teszt (projects.test.ts)

```
it('törli a projektet és az összes kapcsolódó task-ot', async () => {
  const createRes = await request(app)
    .post('/api/projects')
    .set('Authorization', `Bearer ${token}`)
    .send({
      name: 'Törlendő projekt',
      taskGroups: [{
        name: 'Sprint 1', deadline: '2025-07-01',
        tasks: [{ description: 'Törlendő task',
                  subtasks: ['Sub 1'] }],
      }],
    });

  const projectId = createRes.body._id;

  const deleteRes = await request(app)
    .delete(`/api/projects/${projectId}`)
    .set('Authorization', `Bearer ${token}`);

  expect(deleteRes.status).toBe(200);

  const remainingTasks = await Task.find({ project: projectId });
  expect(remainingTasks).toHaveLength(0);
});
```

### 5.2.4. Feladat tesztek

A task tesztek a feladatok létrehozását, lekérését, módosítását és törlését fedik le. Fontos tesztesetek:

- A feladat alapértelmezett státusza `'Open'`.
- A határidő (`deadline`) helyesen tárolódik és visszaadódik.

- A státusz módosítható (pl. `'Open'` → `'InProgress'`).
- Törlés után a feladat nem jelenik meg a projekt feladatlistájában.
- Token nélküli hozzáférés 401-es hibát ad.

### 5.2.5. Alfeladat tesztek

Az alfeladat tesztek hasonló struktúrát követnek, mint a feladattesztek, kiegészítve az XP alapértékének ellenőrzésével (`xpReward: 0`) és az XP mező módosíthatóságával.

### 5.2.6. Service tesztek

**xpService tesztek**: Az `addXP` függvény tesztelése az in-memory adatbázison fut (nem mockolt), így valódi Mongoose dokumentumokon ellenőrizzük a viselkedést. A tesztesetek:

- XP hozzáadásakor az érték helyesen növekszik.
- Küszöbérték elérésénél szintlépés következik be (maradék XP-vel).
- Több szintlépés egyszerre is működik.
- Nulla XP hozzáadása nem változtatja meg az állapotot.
- Nem létező felhasználóra hiba dobódik (`'User not found'`).
- A szint és XP az adatbázisban is perzisztálódik.
- Pontosan a küszöbön lévő XP szintlépést okoz, maradék 0 XP-vel.

Listing 5.3. XP szintlépési teszt (xpService.test.ts)

```
it('több szintet lép egyszerre, ha sok XP érkezik', async () => {
  const user = await createUser(0, 1);
  // 1. szint: 100 XP, 2. szint: 200 XP -> 300 XP kell a 3. szinthez
  const updated = await addXP(user._id.toString(), 350);

  expect(updated.level).toBe(3);
  expect(updated.xp).toBe(50);
});
```

**githubService tesztek**: Az axios modul teljes mocking-jával teszteljük, hogy a service a helyes GitHub API URL-eket hívja-e, a helyes Authorization headert küldi-e el, és a válasz adatait a várt formátumra alakítja-e (pl. `creator` mező a `user.login`-ból).

**logService tesztek**: A `console.log` spy-olásával ellenőrizzük, hogy a naplóbejegyzések a várt formátumban (`[PROJECT id] üzenet`) kerülnek kiírásra. Ez a teszt a log mentési logikát is verifikálja, de mivel a log service az adatbázisba is ír, a tesztek in-memory DB-n futnak.

## 5.3. Frontend tesztelés

### 5.3.1. Eszközök és konfiguráció

A frontend tesztek Vitest tesztelési keretrendszert és React Testing Library-t alkalmaznak. A Vitest Vite-alapú, és natívan kezeli a TypeScript-et és a JSX-et. A React Testing Library a komponensek felhasználói szemszögből való tesztelésére ösztönöz: nem az implementáció részleteit, hanem a látható és interakcióra képes elemeket ellenőrzi.

A tesztek konfigurációja a Vitest konfigurációs fájlban a `jsdom` tesztkörnyezetet alkalmazza, a `@testing-library/jest-dom` pedig bővíti a matchers listát (pl. `toBeInTheDocument`, `toHaveProperty`).

### 5.3.2. Reducer tesztek

A reducer tesztek a legegyszerűbbek: tiszta függvényeket tesztelnek, amelyeknek nincs mellékhatásuk. Az `AppReducer` tesztjei ellenőrzik a `SET_SUBTASKS`, `SET_LOADING` és `SET_SELECTED_PROJECT` action típusokat. A `ViewReducer` tesztjei verifikálják az ismeretlen action típusra adott alapértelmezett viselkedést, és az összes `ViewState` értékkel való kompatibilitást.

Listing 5.4. ViewReducer tesztek (ViewReducer.test.tsx)

```
it('SET_VIEW akció frissíti az activeView-t', () => {
  const action: ViewAction = { type: 'SET_VIEW', payload: 'tasks' };
  const newState = ViewReducer(initialViewState, action);
  expect(newState).toBe('tasks');
});

it('többféle ViewState-et is kezel', () => {
  const states: ViewState[] =
    ['main', 'kanban', 'scrum', 'subtasks', 'log'];
  states.forEach((state) => {
    const newState = ViewReducer(initialViewState,
      { type: 'SET_VIEW', payload: state });
    expect(newState).toBe(state);
  });
});
```

### 5.3.3. Context tesztek

A Context tesztek egyszerű tesztkomponenseket renderelnek a Context Providerbe ágyazva, és ellenőrzik az alapértelmezett állapotot, majd egy gombnyomás szimulációjával a dispatch utáni állapotváltozást.

Listing 5.5. GlobalContext integrációs teszt

```
it('dispatch frissíti a token-t és user-t', () => {
  render(
    <GlobalProvider>
      <TestComponent />
```

```
    </GlobalProvider>,
  );

  fireEvent.click(screen.getByText('Set User'));
  expect(screen.getByText('Token: abc123')).toBeInTheDocument();
});
```

### 5.3.4. Action tesztek

Az action tesztek az axios mock-olásával ellenőrzik, hogy a `fetchTasks` és `addTask` függvények a helyes URL-t hívják-e, a megfelelő headereket küldik-e el, és a szerver válasza alapján a helyes dispatch hívásokat indítják-e el. Hibaeset szimulációjával ellenőrizzük a `SET_ERROR` dispatch-t is.

### 5.3.5. Komponens tesztek

**Sign komponens**: A tesztek ellenőrzik, hogy mindkét form (Register és Login) renderelésre kerül, a form submit az adatokkal meghívja a megfelelő action függvényt, és a loading / error állapotok megjelennek a felületen.

**CreateProject komponens**: A tesztek verifikálják a kezdő lépés megjelenítését, a Manual flow következő lépésre váltást, a contributor hozzáadás működését, és hogy üres contributor nem adható hozzá.

**TasksView és SubTasksView komponensek**: Mockolt globális állapottal ellenőrzik, hogy a feladatok és alfeladatok cím szerint megjelennek-e a nézetben.

**Sidebar és Topbar**: Strukturális tesztek ellenőrzik, hogy a megfelelő számú gomb és a várt HTML elem (`header` szerepkör) megjelenik-e.

## 5.4. Manuális tesztelés

Az automatizált tesztek mellett a rendszer manuális tesztelésen is átesett. Ennek során a következő folyamatokat ellenőriztük végig a böngészőben és fizikai mobil eszközön:

- Teljes regisztráció és bejelentkezés folyamata.
- Projekt létrehozása manuálisan és GitHub importtal.
- Feladatok és alfeladatok hozzáadása, szerkesztése, törlése és befejezése.
- XP jóváírás és szintlépés ellenőrzése a főoldalon és a gamification nézetben.
- Sprint létrehozása, feladat hozzárendelése és sprint törlése.
- Belső levelezés küldése és fogadása két különböző felhasználói fiókkal.
- Aktivitásnapló megjelenítésének ellenőrzése különböző műveletek elvégzése után.
- Profil szerkesztés (felhasználónév, e-mail, jelszó módosítás).
- Mobil alkalmazás teljes funkcionalitásának ellenőrzése Android eszközön (Expo Go).

| Tesztkészlet | Lefedett esetek | Tesztek száma |
|---|---|---|
| auth.test.ts | Regisztráció, bejelentkezés, token validáció | 9 |
| projects.test.ts | CRUD, jogosultság, taskGroup import | 11 |
| tasks.test.ts | CRUD, állapotváltás, határidő kezelés | 8 |
| subtasks.test.ts | CRUD, alapértékek, XP mező | 8 |
| xpService.test.ts | XP hozzáadás, szintlépés, perzisztencia | 8 |
| githubService.test.ts | API hívások, adat transzformáció | 11 |
| logService.test.ts | Naplóformátum, userId kezelés | 5 |
| app.test.ts | Alapvető route ellenőrzés | 1 |
| **Backend összesen** | | **61** |
| Reducer.test.tsx | AppReducer action típusok | 3 |
| ViewReducer.test.tsx | ViewReducer állapotváltás | 3 |
| GlobalContext.test.tsx | Context alapállapot, dispatch | 2 |
| ViewContext.test.tsx | View váltás dispatch-csel | 2 |
| Actions.test.ts | fetchTasks, addTask, hibakezelés | 3 |
| Sign.test.tsx | Form render, submit, loading/error | 4 |
| CreateProject.test.tsx | Lépések, contributor kezelés | 4 |
| TasksView.test.tsx | Feladatlista renderelés | 1 |
| SubTasksView.test.tsx | Alfeladatlista renderelés | 1 |
| Sidebar.test.tsx | Ikonok száma | 1 |
| Topbar.test.tsx | Banner elem | 1 |
| **Frontend összesen** | | **25** |
| **Összes teszteset** | | **86** |

5.1. táblázat. A tesztkészletek összefoglalása

```
Test Files  11 passed (11)
     Tests  47 passed (47)
  Start at  13:06:14
  Duration  5.67s (transform 567ms, setup 1.04s, import 2.44s,
```

5.1. ábra. A frontend unit tesztek futtatásának eredménye (Vitest)

```
Test Suites: 8 passed, 8 total
Tests:       70 passed, 70 total
Snapshots:   0 total
Time:        9.567 s, estimated 11 s
Ran all test suites.
```

5.2. ábra. A backend integrációs tesztek futtatásának eredménye (Jest)

## 5.5. Ismert korlátok és nem tesztelt területek

A tesztelés nem teljes körű, és számos terület kívül esett a tesztlefedettségen. A mail rendszer backend végpontjai nem kaptak dedikált integrációs teszteket, bár a modell és a route struktúrája megegyezik a többi, tesztelt erőforrással. A sprint feladathozzárendelési logika részletes edge case-jei (pl. egy feladat két sprinthez rendelése egymás után) szintén nem kerültek automatizált tesztekbe. A mobil alkalmazás frontend komponenseire nem készültek unit tesztek, elsősorban az Expo és React Native tesztkörnyezet konfigurálásának összetettsége miatt. A teljesítménytesztelés (load testing, stressz teszt) szintén nem volt a dolgozat keretein belül.

Ezek a hiányosságok tudatos döntések eredménye: az elérhető idő és erőforrás keretek között a legkritikusabb üzleti logika (auth, XP, project/task/subtask CRUD) kapta a legnagyobb tesztlefedettséget.

# 6. fejezet

# Összefoglalás

## 6.1. A munka összegzése

A dolgozat egy modern, háromplatformos projektmenedzsment rendszer, az Acxor teljes körű tervezését és implementációját mutatta be. A rendszer egy közös Node.js/Express backend API köré épül, amelyhez egy React alapú webes alkalmazás és egy React Native/Expo alapú mobil alkalmazás csatlakozik kliensként. Mindhárom platform TypeScriptben íródott, ami egységes kódbázis-szemléletet és típusbiztonságot biztosít az egész rendszerben.

A bevezetőben megfogalmazott alapproblémára – miszerint a meglévő projektmenedzsment eszközök vagy túlságosan komplexek, vagy túlságosan egyszerűek, és ritkán párosítják a professzionális funkcionalitást motivációs elemekkel – az Acxor egy letisztult, vizuálisan koherens és gamifikációs elemekkel dúsított megoldást kínál. A koncepciós fejezetben elvégzett piackutatás alapján az Acxor a Trello vizuális egyszerűségét, a JIRA sprint-alapú munkaszervezési képességét és a rendszerek által általában elhanyagolt motivációs réteget ötvözi egyetlen platformban.

A tervezési fejezetben részletesen bemutatásra kerültek az architektúrális döntések: a háromrétegű kliens-szerver modell, a MongoDB adatbázis-séma és a hét Mongoose modell (User, Project, Task, Subtask, Sprint, Log, Mail), a RESTful API tervezési elvei és a JWT alapú hitelesítési folyamat. A felhasználói felület tervezésekor a hangsúly az egységes vizuális identitáson és a három platform közötti konzisztens élményen volt. Külön figyelmet kapott a karbantarthatóság kérdése: a közös állapotkezelési minta, az API contract szerepe és a tudatosan vállalt kódduplikáció mint kezelt technikai adósság mind a tervezési döntések szerves részét képezték.

Az implementációs fejezetben a legfontosabb megvalósítási részletek kerültek terítékre: a backend route-ok és service-ek, az XP és szintlépési algoritmus, a Context API alapú állapotkezelés, a nézetek és komponensek felépítése, valamint a GitHub integrációs megoldás. A mobil alkalmazásnál külön figyelmet kapott az egyedi tab bar megoldás és a fizikai eszközön való fejlesztést lehetővé tevő dinamikus API URL konfiguráció.

A tesztelési fejezetben összesen 86 teszteset kerül bemutatásra: 61 backend integrációs teszt (Jest + Supertest + mongodb-memory-server) és 25 frontend unit teszt (Vitest + React Testing Library). A tesztek lefedik a hitelesítési logikát, a projekt és feladat CRUD műveleteket, az XP service szintlépési algoritmusát, a GitHub service adat-transzformációját és a legfontosabb React komponensek renderelési viselkedését.

## 6.2. Elért eredmények

A fejlesztés során az alábbi főbb eredmények születtek:

- **Teljes körű REST API**: Kilenc végpontcsoport (auth, projects, tasks, subtasks, sprints, logs, users, mails, github) mindegyike authentikált hozzáférést biztosít, és következetesen kezeli a hibákat.
- **Gamifikációs rendszer**: A befejezett feladatokért és alfeladatokért XP jár, amelynek mértéke szintlépéskor csökkenti a fennmaradó értéket, és automatikusan növeli a szintet. A prestige rendszer tovább mélyíti a hosszú távú motivációt.
- **Webes alkalmazás**: Kanban board, Scrum sprint kezelés, Plan nézet projektszintű statisztikákkal, alfeladat kezelés, aktivitásnapló, belső levelezési rendszer, gamifikáció rangsor, wiki és fiókkezelés – mind egyetlen oldalas, Context API alapú alkalmazásban.
- **Mobil alkalmazás**: Az összes fő funkció elérhető Expo Router alapú, tíz tapos navigációval rendelkező React Native alkalmazásban, fizikai Android és iOS eszközön egyaránt.
- **GitHub integráció**: Repozitóriumok listázása és issue-k importálása feladatokként, szerver oldalon proxyzva a GitHub API-n keresztül.
- **Tesztlefedettség**: 86 automatizált teszteset, amelyek a rendszer kritikus üzleti logikáját és API viselkedését verifikálják.
- **Karbantartható architektúra**: Egységes állapotkezelési minta a webes és mobil kliensben, service réteg szétválasztása a route handlerektől, TypeScript típusbiztonság mindhárom platformon, és közös backend mint egyetlen igazságforrás a két kliens számára.

## 6.3. A karbantarthatóság értékelése

A rendszer fejlesztése során a karbantarthatóság nem utólagos szempont volt, hanem tervezési alapelv. Ennek értékelése három dimenzióban végezhető el: a kódbázis struktúrája, a web–mobil összehangoltság és a jövőbeli bővíthetőség szempontjából.

### 6.3.1. Kódbázis struktúrájának karbantarthatósága

A backend kódja jól elkülönített rétegekre épül: a route handlerek kizárólag a HTTP kérések fogadásáért és a válasz küldéséért felelősek, az üzleti logika a service rétegben él (`xpService`, `logService`, `githubService`), az adathozzáférés pedig a Mongoose modelleken keresztül valósul meg. Ez a háromrétegű szétválasztás azt jelenti, hogy például az XP számítási logika megváltoztatásához egyetlen fájlt (`xpService.ts`) kell módosítani, és az összes érintett végpont automatikusan az új logikát alkalmazza. Az ilyen jellegű, jól lokalizálható módosíthatóság a karbantartható szoftverek egyik legfontosabb jellemzője.

A TypeScript használata mindhárom platformon szintén közvetlen karbantarthatósági előnyt jelent: a típusrendszer fordítási időben jelzi az inkompatibilis változásokat,

amelyek dinamikusan típusos kódban csak futásidőben derülnének ki. Ez különösen értékes akkor, amikor az adatmodell változik, és a változásnak mindhárom platformon át kell propagálódnia.

### 6.3.2. Web és mobil kliens összehangoltsága

A webes React alkalmazás és a React Native mobil alkalmazás közötti karbantarthatóság legfőbb biztosítéka a közös állapotkezelési minta. Mindkét kliens ugyanolyan struktúrájú Context-et, Reducer-t és action creator függvényeket alkalmaz, ami azt jelenti, hogy egy fejlesztő, aki az egyik kliensben eligazodik, minimális tanulással képes a másikban is hatékonyan dolgozni. A közös backend API pedig garantálja, hogy az adatmodell változásai mindkét klienshez egyszerre jutnak el, és nem lehetséges, hogy a két kliens eltérő adatszerkezettel dolgozzon.

Ugyanakkor a dolgozat őszintén elismeri a jelenlegi architektúra korlátait is. Az `Actions.ts` fájlban lévő aszinkron action creator logika, a TypeScript interfész definíciók és a hibakezelési konvenciók mindkét kliensben kézzel vannak karbantartva, ami azt jelenti, hogy egy üzleti logika változásakor két helyen kell elvégezni az azonos módosítást. Ez a duplikáció kontrollált és tudatosan vállalt, de hosszú távon – különösen csapatnövekedés esetén – érdemes lenne megszüntetni egy megosztott csomag (shared package) bevezetésével.

### 6.3.3. Összegző értékelés

Összességében az Acxor architektúrája a karbantarthatóság szempontjából egyensúlyt tart a pragmatizmus és az elvek között. A rendszer nem törekszik a tökéletes kódduplikációmentességre, ha az azzal járó komplexitás nem arányos az előnyökkel – de minden fontosabb architekturális döntés mögött felismerhető a karbantarthatósági szempont: a service réteg szétválasztása, a közös állapotkezelési minta, a TypeScript típusbiztonság és a tesztlefedettség együtt olyan rendszert alkotnak, amelyen egy új fejlesztő a belépéstől kezdve hatékonyan tud dolgozni, és amelyen a változtatások hatása előre látható és kontrollálható.

## 6.4. Jövőbeli fejlesztési lehetőségek

A rendszer számos irányban tovább fejleszthető. A legfontosabb jövőbeli lehetőségek az alábbiak:

**Monorepo és megosztott csomag**: A karbantarthatóság legjelentősebb javítása egy monorepo struktúra (pl. Nx vagy Turborepo) bevezetése lenne, amelyen belül egy `shared` csomag tartalmazná a TypeScript interfész definíciókat, az action típusokat és a közös üzleti logikát. Ez megszüntetné a jelenlegi kódduplikációt a webes és mobil kliens között, és garantálná, hogy az adatmodell változásai automatikusan és konzisztensen érvényre jutnak mindkét kliensben – jelentősen csökkentve az összehangolási hibák kockázatát.

**API contract és típusgenerálás**: A backend TypeScript interfészeit jelenleg kézzel másolják a kliensek. Egy OpenAPI/Swagger séma alapú típusgenerálási pipeline (pl. `openapi-typescript` eszközzel) lehetővé tenné, hogy a backend interfészei automatikusan generált típusként legyenek elérhetők mindkét kliensben. Ez az API contract

szinkronizálási problémát teljesen megszüntetné: ha a backend változik, a következő generálási lépés után a kliensek azonnal látják a típusinkompatibilitást.

**Drag-and-drop a Kanban boardon**: A jelenlegi implementáció statikus kártyákat jelenít meg. A feladatok húzással való áthelyezése oszlopok között – és ezzel egyidejűleg az állapot módosítása – jelentősen javítaná a felhasználói élményt. Ez webes oldalon például a `@dnd-kit` könyvtárral valósítható meg.

**Valós idejű frissítések**: A jelenlegi rendszer lekérdezés alapú: az adatok akkor frissülnek, ha a felhasználó navigál vagy manuálisan frissít. WebSocket vagy Server-Sent Events alapú megoldással (pl. Socket.io) a csapat tagjai valós időben láthatnák egymás változtatásait.

**E-mail értesítések**: Feladat hozzárendeléskor vagy határidő közeledésekor automatikus e-mail értesítés küldése (pl. Nodemailer és egy SMTP szolgáltató segítségével).

**Sprint törlés és feladat konzisztencia**: Jelenleg sprint törlésekor a hozzá rendelt feladatok `sprint` mezője nem törlődik automatikusan a backendben, ami konzisztencia-problémát okozhat. Ezt egy MongoDB aggregációs pipeline-nal vagy Mongoose middlewarerel lehetne kezelni.

**Szerepkör-alapú jogosultságkezelés**: A jelenlegi rendszer megkülönböztet adminisztrátor és közreműködő szerepköröket, de a végrehajtási logika nem minden végponton érvényes egységesen. Egy részletesebb middleware-alapú permission réteg pontosabb hozzáférésvezérlést biztosítana.

**Prestige és class badge rendszer teljes kiépítése**: A gamifikációs tervezés tartalmaz egy class badge rendszert (szintenként három osztály közül lehet választani), amelynek UI-ja elkészült, de a mögötte lévő adatmodell (választott osztály tárolása) még nem implementált.

**Tesztlefedettség bővítése**: A mail rendszer backend integrációs tesztjei, a sprint tesztek és a mobil alkalmazás komponens tesztjei mind hasznos kiegészítések lennének a jövőbeli fejlesztések során. A mobil tesztelés hiánya elsősorban az Expo tesztkörnyezet konfigurálásának összetettsége miatt maradt el, de egy `jest-expo` alapú konfiguráció ezt orvosolhatná.

**Swagger/OpenAPI dokumentáció**: Az API végpontok automatikus dokumentálása swagger-jsdoc vagy tsoa segítségével interaktív, böngészhető dokumentációt biztosítana a jövőbeli integrációs partnerek és fejlesztők számára.

## 6.5. Zárszó

Az Acxor fejlesztése megmutatta, hogy egy közepes komplexitású, háromplatformos alkalmazás megtervezése és implementálása egyetlen fejlesztőként is megvalósítható, ha a technológiai döntések jól vannak meghozva és a kódstruktúra kezdettől fogva gondosan ki van alakítva. A közös backend API, az újrafelhasznált üzleti logika és a konzisztens állapotkezelési minta lehetővé tette, hogy a három platform – webes, mobil és szerver – szorosan összehangoltan működjön.

A karbantarthatóság szempontjából a projekt egyik legfontosabb tanulsága az, hogy a webes és mobil kliens közötti konzisztencia nem jön magától: tudatos tervezési döntéseket igényel, amelyeket már az architektúra kialakításakor meg kell hozni. A közös állapotkezelési minta és a közös backend API mint egyetlen igazságforrás megfelelő alapot adott ehhez, ugyanakkor a kódduplikáció és az API contract manuális szinkronizálásának kérdése rámutat arra, hogy egy megosztott csomag alapú megközelítés a rendszer következő fejlődési lépése lehet.

A dolgozat elkészítése során személyes fejlődés is tapasztalható volt: a TypeScript alkalmazása kikényszerítette a pontosabb gondolkodást az adatstruktúrák körül, az automatizált tesztelés megkövetelte a moduláris kódszervezést, a mobil fejlesztés pedig új kihívásokat hozott a navigáció, a natív UI komponensek és a fizikai eszközön való fejlesztés terén.

Az Acxor jelenlegi formájában egy funkcionális, tesztelt és bővíthető alapot jelent. A jövőbeli fejlesztési lehetőségek gazdag tárháza nyitva áll, és a rendszer architektúrája – különösen a service réteg szétválasztása, a TypeScript típusbiztonság és az egységes állapotkezelési minta – ezeket a bővítményeket lehetővé teszi anélkül, hogy az alap kódbázist drasztikusan át kellene alakítani.

# CD Használati útmutató

Ennek a címe lehet például *A mellékelt CD tartalma* vagy *Adathordozó használati útmutató* is.

Ez jellemzően csak egy fél-egy oldalas leírás. Arra szolgál, hogy ha valaki kézhez kapja a szakdolgozathoz tartozó CD-t, akkor tudja, hogy mi hol van rajta. Jellemzően elég csak felsorolni, hogy milyen jegyzékek vannak, és azokban mi található. Az elkészített programok telepítéséhez, futtatásához tartozó instrukciók kerülhetnek ide.

A CD lemezre mindenképpen rá kell tenni

- a dolgozatot egy `dolgozat.pdf` fájl formájában,
- a LaTeX forráskódját a dolgozatnak,
- az elkészített programot, fontosabb futási eredményeket (például ha kép a kimenet),
- egy útmutatót a CD használatához (ami lehet ez a fejezet külön PDF-be vagy MarkDown fájlként kimentve).